

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

26 个 Kernel · 10 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 6 月 24 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (共 26 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 2D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89 //
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPs
102 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
103 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
104 //
105 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
106 //
107 // =====
108 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
109 // =====
110 // 面试要点:
111 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
112 //   memory
113 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
114 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
115 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
116 //
117 #include <algorithm>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127
128 #define WARP_SIZE 32
129 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
130 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
131 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
132
133 // =====
134 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
135 // =====
136
137 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
138 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
139 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
140 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
141 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
142 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
143 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
144 //
145 // 初始: 每个 lane 持有自己的值 v0..v7
146 // lane:  0    1    2    3    4    5    6    7
147 // val:  v0   v1   v2   v3   v4   v5   v6   v7
148 //
149 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
150 //
151 //      ┌──────────┐
152 // 对:   (0,4) (1,5) (2,6) (3,7)
153 //
154 // lane:  0    1    2    3    4    5    6    7
155 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
156 //
157 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
158 //
159 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
160 // 对:   (0,2) (1,3) (4,6) (5,7)
161 //      ─── 前4个一组 ───      ─── 后4个一组 ───
162 //
163 // lane:  0    1    2    3    4    5    6    7
164 // val:  $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$  (每lane持有前一轮2个值的和再加本轮配对)
165 //      = v0+v2+v4+v6 ... (逐步归约)
166 //
167 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
168 //
169 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
170 // 对:   (0,1) (2,3) (4,5) (6,7)
171 //
172 // lane:  0    1    2    3    4    5    6    7
173 // val:  $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
174 //
175 // mask=0: 循环终止, 归约完成。
176 //
177 // 关键性质:
178 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
179 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
180 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
181 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
182 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
183 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
184
185 template <const int kWarpSize = WARP_SIZE, typename T = float>
186 __device__ __forceinline__ T warp_reduce_sum(T val) {
187 #pragma unroll
188     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
189         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
190     }
191     return val;
192 }

```

```

188 }
189
190 // Warp Reduce Max — generic
191 template <const int kWarpSize = WARP_SIZE, typename T = float>
192 __device__ __forceinline__ T warp_reduce_max(T val) {
193 #pragma unroll
194     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
195         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
196     }
197     return val;
198 }
199
200 // =====
201 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
202 // =====
203
204 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
205 // 两级归约流程:
206 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
207 // 2. warp leader (lane=0) 写入 shared memory
208 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
209 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
210 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
211 template <const int NUM_THREADS = 256>
212 __device__ float block_reduce_sum(float val) {
213     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
214     int warp = threadIdx.x / WARP_SIZE;
215     int lane = threadIdx.x % WARP_SIZE;
216     static __shared__ float shared[NUM_WARPS];
217
218     float value = warp_reduce_sum<WARP_SIZE>(val);
219     if (lane == 0)
220         shared[warp] = value;
221     __syncthreads();
222     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
223     value = warp_reduce_sum<NUM_WARPS>(value);
224     // 关键: broadcast 结果到所有线程, 后续用 result
225     // 做除法等操作时所有线程都能拿到
226     value = __shfl_sync(0xffffffff, value, 0, 32);
227     return value;
228 }
229
230 // Block Reduce Max — FP32 (增强版, 带 broadcast)
231 template <const int NUM_THREADS = 256>
232 __device__ float block_reduce_max(float val) {
233     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
234     int warp = threadIdx.x / WARP_SIZE;
235     int lane = threadIdx.x % WARP_SIZE;
236     static __shared__ float shared[NUM_WARPS];
237
238     float value = warp_reduce_max<WARP_SIZE>(val);
239     if (lane == 0)
240         shared[warp] = value;
241     __syncthreads();
242     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
243     value = warp_reduce_max<NUM_WARPS>(value);
244     value = __shfl_sync(0xffffffff, value, 0, 32);
245     return value;
246 }
247
248 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
249 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
250 // 跨 block 求和的常见模式, 适合 N 较大时使用;

```

```

251 // Grid: ((N + 255) / 256, 1, 1)
252 // Block: (256, 1, 1)
253 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
254 template <const int NUM_THREADS = 256>
255 __global__ void block_reduce_all(float *a, float *y, int N) {
256     int tid = threadIdx.x;
257     int idx = blockIdx.x * NUM_THREADS + tid;
258     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
259     __shared__ float reduce_smem[NUM_WARPS];
260
261     float sum = (idx < N) ? a[idx] : 0.0f;
262     int warp = tid / WARP_SIZE;
263     int lane = tid % WARP_SIZE;
264
265     sum = warp_reduce_sum<WARP_SIZE>(sum);
266     if (lane == 0)
267         reduce_smem[warp] = sum;
268     __syncthreads();
269
270     sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
271     if (warp == 0)
272         sum = warp_reduce_sum<NUM_WARPS>(sum);
273     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
274         atomicAdd(y, sum);
275 }
276
277 // ---- Dot Product: y = sum(a[i] * b[i]) ----
278 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
279 // Grid: ((N + 255) / 256, 1, 1)
280 // Block: (256, 1, 1)
281 // source: LeetCUDA/kernels/dot-product/dot_product.cu
282 template <const int NUM_THREADS = 256>
283 __global__ void dot(float *a, float *b, float *y, int N) {
284     int tid = threadIdx.x;
285     int idx = blockIdx.x * NUM_THREADS + tid;
286     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
287     __shared__ float reduce_smem[NUM_WARPS];
288
289     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
290     int warp = tid / WARP_SIZE;
291     int lane = tid % WARP_SIZE;
292
293     prod = warp_reduce_sum<WARP_SIZE>(prod);
294     if (lane == 0)
295         reduce_smem[warp] = prod;
296     __syncthreads();
297
298     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
299     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
300         prod = warp_reduce_sum<NUM_WARPS>(prod);
301     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
302         atomicAdd(y, prod);
303 }
304
305 // Dot Product + float4
306 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素，float4向量化后每个线程处理4元素
307 // Block: (64, 1, 1), 256/4=64
308 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
309 // source: LeetCUDA/kernels/dot-product/dot_product.cu
310 template <const int NUM_THREADS = 256 / 4>
311 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
312     int tid = threadIdx.x;
313     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;

```

```

314 constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
315 __shared__ float reduce_smem[NUM_WARPS];
316
317 float4 reg_a = FLOAT4(a[idx]);
318 float4 reg_b = FLOAT4(b[idx]);
319 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
320                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
321                  : 0.0f;
322 int warp = tid / WARP_SIZE;
323 int lane = tid % WARP_SIZE;
324
325 prod = warp_reduce_sum<WARP_SIZE>(prod);
326 if (lane == 0)
327     reduce_smem[warp] = prod;
328 __syncthreads();
329
330 prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
331 if (warp == 0)
332     prod = warp_reduce_sum<NUM_WARPS>(prod);
333 if (tid == 0)
334     atomicAdd(y, prod);
335 }
336
337
338 // =====
339 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
340 // =====
341 // 面试要点:
342 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
343 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
344 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
345
346 // ---- ReLU: y = max(0, x) ----
347 // Grid: ((N + 255) / 256, 1, 1)
348 // Block: (256, 1, 1)
349 // source: LeetCUDA/kernels/relu/relu.cu
350 __global__ void relu(float *x, float *y, int N) {
351     int idx = blockIdx.x * blockDim.x + threadIdx.x;
352     if (idx < N)
353         y[idx] = fmaxf(0.0f, x[idx]);
354 }
355
356 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
357 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
358 // Grid: ((N + 255) / 256, 1, 1)
359 // Block: (64, 1, 1)
360 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
361 // source: LeetCUDA/kernels/relu/relu.cu
362 __global__ void relu_vec4(float *x, float *y, int N) {
363     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
364     if (idx < N) {
365         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
366         float4 reg_y;
367         reg_y.x = fmaxf(0.0f, reg_x.x);
368         reg_y.y = fmaxf(0.0f, reg_x.y);
369         reg_y.z = fmaxf(0.0f, reg_x.z);
370         reg_y.w = fmaxf(0.0f, reg_x.w);
371         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
372     }
373 }
374
375 // ---- Elementwise Add: c = a + b ----
376 // Grid: ((N + 255) / 256, 1, 1)

```

```

377 // Block: (256, 1, 1)
378 // source: LeetCUDA/kernels/elementwise/elementwise.cu
379 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
380     int idx = blockIdx.x * blockDim.x + threadIdx.x;
381     if (idx < N)
382         c[idx] = a[idx] + b[idx];
383 }
384
385 // Elementwise Add + float4 向量化
386 // Grid: ((N + 255) / 256, 1, 1)
387 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
388 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
389 // source: LeetCUDA/kernels/elementwise/elementwise.cu
390 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
391     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
392     if ((idx + 3) < N) {
393         float4 reg_a = FLOAT4(a[idx]);
394         float4 reg_b = FLOAT4(b[idx]);
395         float4 reg_c;
396         reg_c.x = reg_a.x + reg_b.x;
397         reg_c.y = reg_a.y + reg_b.y;
398         reg_c.z = reg_a.z + reg_b.z;
399         reg_c.w = reg_a.w + reg_b.w;
400         FLOAT4(c[idx]) = reg_c;
401     } else if (idx < N) {
402         for (int i = 0; (idx + i) < N; i++) {
403             c[idx + i] = a[idx + i] + b[idx + i];
404         }
405     }
406 }
407
408 // ---- Histogram: y[a[i]]++ ----
409 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
410 // Grid: ((N + 255) / 256, 1, 1)
411 // Block: (256, 1, 1)
412 // source: LeetCUDA/kernels/histogram/histogram.cu
413 __global__ void histogram(int *a, int *y, int N) {
414     int idx = blockIdx.x * blockDim.x + threadIdx.x;
415     if (idx < N)
416         atomicAdd(&(y[a[idx]]), 1);
417 }
418
419 // =====
420 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
421 // =====
422 // 面试要点:
423 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
424 //     online (1-pass, 增量更新)
425 //   - Online Softmax 是 FlashAttention 的数学基础
426 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
427 //     + variance)
428 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
429
430 // =====
431 // Phase 3a: Softmax — 三级递进 (面试核心考点)
432 // =====
433 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
434 // 回答线索: naive(溢出) → safe → online
435
436 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
437 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
438 // 问题: x 值很大时 exp(x) 溢出为 inf
439 template <const int NUM_THREADS = 256>

```



```

440 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
441 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
442 // source: LeetCUDA/kernels/softmax/softmax.cu
443 __global__ void softmax_per_token(float *x, float *y, int N) {
444     const int tid = threadIdx.x;
445     const int idx = blockIdx.x * blockDim.x + tid;
446
447     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
448     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
449     if (idx < N)
450         y[idx] = exp_val / exp_sum;
451 }
452
453 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
454 // 面试重点: 为什么 Softmax 需要 Safe?
455 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
456 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
457 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
458 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
459 // Grid: (S, 1, 1)
460 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
461 // source: LeetCUDA/kernels/softmax/softmax.cu
462 template <const int NUM_THREADS = 256>
463 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
464     const int tid = threadIdx.x;
465     const int idx = blockIdx.x * blockDim.x + tid;
466
467     // Pass 1: block reduce max — 找最大值
468     float val = (idx < N) ? x[idx] : -FLT_MAX;
469     float max_val = block_reduce_max<NUM_THREADS>(val);
470
471     // Pass 2: exp(x - max) → block reduce sum
472     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
473     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
474
475     if (idx < N)
476         y[idx] = exp_val / exp_sum;
477 }
478
479 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
480 // 面试重点: Online Softmax 为什么重要?
481 //   - Safe Softmax 需要 3 遍遍历: 找 max → exp+sum → 除法
482 //   - Online Softmax 用递推公式合并为 1 遍遍历
483 //   - 核心思想: 处理新元素时, 用新旧 max 的差值 rescale 旧 denominator
484 //   - 正是 FlashAttention 中 "online rescaling":
485 //       
$$O_{\text{new}} = \text{diag}(\exp(m_{\text{old}} - m_{\text{new}})) * O_{\text{old}} + \exp(m_{\text{cur}} - m_{\text{new}}) * P@V$$

486 // 算法参考: "Online normalizer calculation for softmax" (arXiv:1805.02867)
487
488 // ---- Online Softmax 辅助结构 ----
489 // MD struct: 存储 running max (m) 和 running denominator (d = Σexp(x - m))
490 //
491 // 两种使用场景 (公式不同, 但结果等价):
492 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
493 //       m_new = max(m_old, x_i)
494 //       d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
495 //   2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
496 //       m = max(m1, m2)
497 //       d = d1*exp(m1-m) + d2*exp(m2-m)
498 //       当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
499 //
500 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
501 struct __align__(8) MD {
502     float m; // running max

```

```

503     float d; // running denominator (sum of exp(x - max))
504 };
505
506 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
507 //
508 // 不同于单元元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)) ,
509 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
510 // (m1,d1): max 和  $\sum \exp(x_j - m1)$ , 覆盖集合 S1
511 // (m2,d2): max 和  $\sum \exp(x_k - m2)$ , 覆盖集合 S2
512 //
513 // 合并公式 (对称, 不依赖"新/旧"概念) :
514 // m = max(m1, m2)
515 // d = d1*exp(m1-m) + d2*exp(m2-m) ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
516 //
517 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
518 template <const int kWarpSize = WARP_SIZE>
519 __device__ __forceinline__ MD warp_reduce_md_op(MD value) {
520 #pragma unroll
521     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
522         MD other;
523         other.m = __shfl_xor_sync(0xffffffff, value.m, mask, kWarpSize);
524         other.d = __shfl_xor_sync(0xffffffff, value.d, mask, kWarpSize);
525
526         bool value_bigger = (value.m > other.m);
527         MD bigger_m = value_bigger ? value : other;
528         MD smaller_m = value_bigger ? other : value;
529
530         // d_bigger 无需 rescale (其基准 max 已是全局 max) ,
531         // d_smaller 需 rescale: exp(m_smaller - m_bigger)
532         value.d = bigger_m.d + smaller_m.d * __expf(smaller_m.m - bigger_m.m);
533         value.m = bigger_m.m;
534     }
535     return value;
536 }
537
538 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
539 // Grid: (S, 1, 1)
540 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
541 // source: LeetCUDA/kernels/softmax/softmax.cu
542 template <const int NUM_THREADS = 256>
543 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
544     int local_tid = threadIdx.x;
545     int global_tid = blockIdx.x * NUM_THREADS + threadIdx.x;
546     const int WARP_NUM = NUM_THREADS / WARP_SIZE;
547     int warp_id = local_tid / WARP_SIZE;
548     int lane_id = local_tid % WARP_SIZE;
549
550     // 初始化: 每个线程持有一个 (max, denom) 对
551     MD val;
552     val.m = global_tid < N ? x[global_tid] : -FLT_MAX;
553     val.d = global_tid < N ? 1.0f : 0.0f;
554
555     // Block reduce: warp_reduce_md_op 在归约中自动更新 m 和 d
556     __shared__ MD shared[WARP_NUM];
557     MD res = warp_reduce_md_op<WARP_SIZE>(val);
558
559     if (lane_id == 0)
560         shared[warp_id] = res;
561     __syncthreads();
562
563     // 第二级归约: warp0 收集各 warp 结果再做一次 md_op (复用 block_reduce 模式)
564     // 只有 local_tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个 block 可见
565     if (local_tid < WARP_SIZE) {

```

```

566 MD block_res =
567     local_tid < WARP_NUM ? shared[local_tid] : MD{-FLT_MAX, 0.0f};
568 block_res = warp_reduce_md_op<WARP_NUM>(block_res);
569 if (local_tid == 0) {
570     shared[0] = block_res;
571 }
572 }
573 __syncthreads();
574
575 // 用全局 max 和 denom 做最终 softmax
576 MD final_res = shared[0];
577 float d_total_inverse = __fdivdef(1.0f, final_res.d);
578 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
579 if (global_tid < N) {
580     y[global_tid] = __expf(x[global_tid] - final_res.m) * d_total_inverse;
581 }
582 }
583
584 // =====
585 // Phase 3b: RMS Normalization (1-pass reduce)
586 // =====
587 // 面试要点:
588 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
589 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
590 //   - Llama 系列使用 RMS Norm
591 //   - grid(N, K/K), block(K): 一行一个 block
592 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
593 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
594 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
595 template <const int NUM_THREADS = 128>
596 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
597     int tid = threadIdx.x;
598     int bid = blockIdx.x;
599     int idx = bid * blockDim.x + threadIdx.x;
600     const float epsilon = 1e-5f;
601
602     __shared__ float s_variance;
603     float value = (idx < N * K) ? x[idx] : 0.0f;
604     float variance = value * value;
605     variance = block_reduce_sum<NUM_THREADS>(variance);
606     if (tid == 0)
607         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
608     __syncthreads();
609     if (idx < N * K)
610         y[idx] = (value * s_variance) * g;
611 }
612
613 // RMS Norm + float4
614 // Grid: (N, 1, 1)
615 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
616 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
617 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
618 template <const int NUM_THREADS = 128 / 4>
619 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
620     int tid = threadIdx.x;
621     int bid = blockIdx.x;
622     int idx = (bid * blockDim.x + threadIdx.x) * 4;
623     const float epsilon = 1e-5f;
624
625     __shared__ float s_variance;
626     float4 reg_x = FLOAT4(x[idx]);
627     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
628         reg_x.z * reg_x.z + reg_x.w * reg_x.w)

```

```

629         : 0.0f;
630     variance = block_reduce_sum<NUM_THREADS>(variance);
631     if (tid == 0)
632         s_variance = rsqrtf(variance / (float)K + epsilon);
633     __syncthreads();
634     float4 reg_y;
635     reg_y.x = reg_x.x * s_variance * g;
636     reg_y.y = reg_x.y * s_variance * g;
637     reg_y.z = reg_x.z * s_variance * g;
638     reg_y.w = reg_x.w * s_variance * g;
639     if (idx < N * K)
640         FLOAT4(y[idx]) = reg_y;
641 }
642
643 // =====
644 // Phase 3c: Layer Normalization (2-pass reduce)
645 // =====
646 // 面试要点:
647 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
648 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
649 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
650 // Grid: (N, 1, 1), 一行一个 block
651 // Block: (128, 1, 1), NUM_THREADS=K=128
652 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
653 template <const int NUM_THREADS = 128>
654 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
655     int tid = threadIdx.x;
656     int bid = blockIdx.x;
657     int idx = bid * blockDim.x + threadIdx.x;
658     const float epsilon = 1e-5f;
659
660     __shared__ float s_mean;
661     __shared__ float s_variance;
662     float value = (idx < N * K) ? x[idx] : 0.0f;
663
664     // Pass 1: compute mean
665     float sum = block_reduce_sum<NUM_THREADS>(value);
666     if (tid == 0)
667         s_mean = sum / (float)K;
668     __syncthreads(); // 必须等待 s_mean 对所有线程可见
669
670     // Pass 2: compute variance = (x - mean)2
671     float variance = (value - s_mean) * (value - s_mean);
672     variance = block_reduce_sum<NUM_THREADS>(variance);
673     if (tid == 0)
674         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
675     __syncthreads(); // 必须等待 s_variance 对所有线程可见
676
677     if (idx < N * K)
678         y[idx] = ((value - s_mean) * s_variance) * g + b;
679 }
680
681 // Layer Norm + float4
682 // Grid: (N, 1, 1)
683 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
684 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
685 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
686 template <const int NUM_THREADS = 128 / 4>
687 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N,
688                                 int K) {
689     int tid = threadIdx.x;
690     int bid = blockIdx.x;
691     int idx = (bid * blockDim.x + threadIdx.x) * 4;

```

```

692 const float epsilon = 1e-5f;
693
694 __shared__ float s_mean;
695 __shared__ float s_variance;
696 float4 reg_x = FLOAT4(x[idx]);
697 float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
698
699 float sum = block_reduce_sum<NUM_THREADS>(value);
700 if (tid == 0)
701     s_mean = sum / (float)K;
702 __syncthreads();
703
704 float4 reg_x_hat;
705 reg_x_hat.x = reg_x.x - s_mean;
706 reg_x_hat.y = reg_x.y - s_mean;
707 reg_x_hat.z = reg_x.z - s_mean;
708 reg_x_hat.w = reg_x.w - s_mean;
709 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
710                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
711 variance = block_reduce_sum<NUM_THREADS>(variance);
712 if (tid == 0)
713     s_variance = rsqrtf(variance / (float)K + epsilon);
714 __syncthreads();
715
716 float4 reg_y;
717 reg_y.x = reg_x_hat.x * s_variance * g + b;
718 reg_y.y = reg_x_hat.y * s_variance * g + b;
719 reg_y.z = reg_x_hat.z * s_variance * g + b;
720 reg_y.w = reg_x_hat.w * s_variance * g + b;
721 if (idx < N * K)
722     FLOAT4(y[idx]) = reg_y;
723 }
724
725 // =====
726 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
727 // =====
728 // 面试要点:
729 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
730 //       [x1']   [cos(θ)  -sin(θ)] [x1]
731 //       [x2'] = [sin(θ)   cos(θ)] [x2]
732 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
733 //   - token_pos = idx / N: token 在序列中的位置
734 //   - token_idx = idx % N: token 内的维度对索引
735 //   - 输入 [seq_len, hidden_size], 输出同形状
736
737 // Grid: ((seq_len * N + 255) / 256, 1, 1)
738 // Block: (256, 1, 1)
739 // source: LeetCUDA/kernels/rope/rope.cu
740 __global__ void rope(float *x, float *out, int seq_len, int N) {
741     int idx = blockIdx.x * blockDim.x + threadIdx.x;
742     float x1 = x[idx * 2];
743     float x2 = x[idx * 2 + 1];
744     int token_pos = idx / N; // 序列位置
745     int token_idx = idx % N; // 维度对索引
746
747     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
748     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
749     float sin_v = sinf(token_pos * exp_v);
750     float cos_v = cosf(token_pos * exp_v);
751
752     // 2D 旋转
753     float out1 = x1 * cos_v - x2 * sin_v;
754     float out2 = x1 * sin_v + x2 * cos_v;

```

```

755     out[idx * 2] = out1;
756     out[idx * 2 + 1] = out2;
757 }
758
759 // =====
760 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
761 // =====
762 // 面试要点 (Bank Conflict 专题) :
763 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
764 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
765 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
766 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
767 //
768 // 转置的四步演进:
769 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
770 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
771 //   BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
772 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
773 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
774
775 // 每个线程处理 1 个元素, block(16,16)
776 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
777 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
778 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
779 // Block: (16, 16, 1)
780 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
781 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
782     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
783     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
784     if (r < row && c < col) {
785         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
786         y[c * row + r] = x[r * col + c];
787     }
788 }
789
790 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
791 // Block: (16, 16, 1)
792 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
793 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
794 __global__ void mat_transpose_padded(
795     float *x, float *y, const int row, const int col) {
796     const int tx = threadIdx.x, ty = threadIdx.y;
797
798     constexpr int TILE = 16;
799     constexpr int PAD = 1;
800     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
801     __shared__ float tile[TILE * 4][TILE + PAD];
802
803     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
804     const int x_c = blockIdx.x * TILE + tx;
805     const int x_r = blockIdx.y * TILE + ty;
806
807     if (x_r * 4 < row && x_c < col) {
808         // Step 1: 4 rows × 1 col → smem (row-major);
809 #pragma unroll
810         for (int i = 0; i < 4; ++i) {
811             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
812         }
813         __syncthreads();
814
815         // Step 2: Transposed read from smem
816         float4 reg_trans;
817         reg_trans.x = tile[tx * 4 + 0][ty];

```

```

818     reg_trans.y = tile[tx * 4 + 1][ty];
819     reg_trans.z = tile[tx * 4 + 2][ty];
820     reg_trans.w = tile[tx * 4 + 3][ty];
821
822     // y 空间坐标: y_r = x 的列, y_c = x 的行
823     const int y_r = blockIdx.x * TILE + ty;          // = x col
824     const int y_c = (blockIdx.y * TILE + tx) * 4;    // = x row
825
826     // Coalesced write: y[y_r][y_c .. y_c+3]
827     FLOAT4(y[y_r * row + y_c]) = reg_trans;
828 }
829 }
830
831 // =====
832 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
833 // =====
834 // 面试要点:
835 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
836 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
837 //   - 不同 K 值对应不同分块策略:
838 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
839 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
840 //       K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
841 //
842 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
843
844 // ---- SGEMV K32: 基础 warp-per-row ----
845 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
846 // grid(M/4), 每个 warp 负责一行
847 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
848 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
849 // Block: (32, 4, 1), 每行 1 warp
850 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
851 // source: LeetCUDA/kernels/sgemv/sgemv.cu
852 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
853     int tx = threadIdx.x; // 0~31
854     int ty = threadIdx.y; // 0~3
855     int bx = blockIdx.x;
856     int lane = tx % WARP_SIZE; // 0~31
857     int m = bx * blockDim.y + ty; // 全局行号
858     if (m < M) {
859         float sum = 0.0f;
860         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
861         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
862         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
863 #pragma unroll
864         for (int w = 0; w < NUM_ITERS; ++w) {
865             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
866             int k = w * WARP_SIZE + lane;
867             sum += a[m * K + k] * x[k];
868         }
869         sum = warp_reduce_sum<WARP_SIZE>(sum);
870         // 每个 warp 处理一行, lane 0 写回结果
871         if (lane == 0)
872             y[m] = sum;
873     }
874 }
875
876 // ---- SGEMV K128: float4 向量化 ----
877 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
878 // Grid: ((M + 3) / 4, 1, 1)
879 // Block: (32, 4, 1)
880 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐

```



```

881 // source: LeetCUDA/kernels/sgemv/sgemv.cu
882 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
883     int tx = threadIdx.x; // 0~31
884     int ty = threadIdx.y; // 0~3
885     int bx = blockIdx.x;
886     int lane = tx % WARP_SIZE; // 0~31
887     int m = blockDim.y * bx + ty;
888
889     if (m < M) {
890         float sum = 0.0f;
891         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
892         const int NUM_ITERS = ((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
893         #pragma unroll
894         for (int w = 0; w < NUM_ITERS; ++w) {
895             int k = (w * WARP_SIZE + lane) * 4;
896             float4 reg_x = FLOAT4(x[k]);
897             float4 reg_a = FLOAT4(a[m * K + k]);
898             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
899                 reg_a.w * reg_x.w);
900         }
901         sum = warp_reduce_sum<WARP_SIZE>(sum);
902         if (lane == 0)
903             y[m] = sum;
904     }
905 }
906
907 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
908 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
909 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
910 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
911 // Block: (32, 4, 1)
912 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
913 // source: LeetCUDA/kernels/sgemv/sgemv.cu
914 template <const int ROW_PER_WARP = 2>
915 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
916     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
917     int tx = threadIdx.x;
918     int ty = threadIdx.y;
919     int bx = blockIdx.x;
920     int lane = tx % WARP_SIZE;
921     int k = lane % K_WARP_SIZE; // 0~15
922     int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
923     if (m < M) {
924         float sum = A[m * K + k] * x[k];
925         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
926         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
927         // 注意: 判断条件是 k == 0, 不是 lane == 0!
928         if (k == 0)
929             y[m] = sum;
930     }
931 }
932
933 // =====
934 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
935 // =====
936 // 面试要点 (GEMM 优化五层金字塔):
937 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
938 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
939 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
940 //   load/store 指令数 Level 4 — Tensor Core (MMA
941 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
942 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
943 //

```



```

944 // 计算密度递进:
945 //   Level 1: AI  $\approx$  B_K / (2*sizeof)  $\approx$  32/8 = 4  $\rightarrow$  仍是 memory-bound
946 //   Level 2: AI  $\approx$  TM*TN*B_K / (2*sizeof)  $\approx$  8*8*8/8 = 64  $\rightarrow$  compute-bound
947 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp  $\rightarrow$  大幅提升吞吐
948
949 // =====
950 // Phase 7a: SGEMM + HGEMM (非 Tensor Core 路径)
951 // =====
952
953 // ---- Level 1: SGEMM — Block Tile 32x32 + K Tile 32 ----
954 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
955 // C = A x B, C[M, N] = A[M, K] x B[K, N]
956 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
957 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
958 // Block: (32, 32, 1), 1024 线程
959 // source: LeetCUDA/kernels/sgemm/sgemm.cu
960 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
961     constexpr int BM = 32; // vec 版: 32x4 = 128
962     constexpr int BN = 32; // vec 版: 32x4 = 128
963     constexpr int BK = 32;
964     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
965
966     int bx = blockIdx.x;
967     int by = blockIdx.y;
968     int tx = threadIdx.x;
969     int tid = threadIdx.y * blockDim.x + tx;
970
971     // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
972     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载; vec 版: a_m = tid / (32 / 4), row 0~127, [128x32]
973     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载; vec 版: a_k = (tid % (32 / 4)) * 4, t 0~7, col
974     // 0~31, 每个线程加载 4 个元素, 8x4 = 32
975     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载; vec 版: b_k = tid / 32, row 0~31, [32x128]
976     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载; vec 版: b_n = (tid % 32) * 4, t 0~31, col 0~127, 每
977     // 个线程加载 4 个元素, 32x4 = 128
978     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row; vec 版: load_smem_a_m, 0~127, 0, 1, 2, ... (连续)
979     // [128x128]
980     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col; vec 版: load_smem_b_n, 0~127, 0, 4, 8, ... (间隔)
981
982     float sum = 0.f; // 遍历完整的K, slice K; vec 版: sum[4][4] = 0.f; 每个线程处理4x4的大小, 才能保证32x32线程处理[32
983     // x4,32x4]=[128x128]大小
984     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
985         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
986         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
987         // vec 版: FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr])
988         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
989         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
990         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
991         // vec 版: FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
992         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
993         __syncthreads(); // 确保整个 smem tile 加载完毕
994     }
995
996     #pragma unroll
997     for (int k = 0; k < BK; ++k) {
998         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
999         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1000         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1001     }
1002     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1003
1004     int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1005     int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1006     int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1007     c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]

```

```

1003 }
1004
1005 // ---- Level 1+: SGEMM Vec4 — Block Tile 128×128 + K Tile 32 + Thread Tile 4×4 ----
1006 // 在 Level 1 基础上引入两层优化:
1007 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1008 // 2) Thread Tile 4×4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1009 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1010 // C = A × B, C[M, N] = A[M, K] × B[K, N], A/B 均 row-major
1011 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4×4=16 个 C 元素
1012 // 1024 × 16 = 16384 = 128 × 128 ✓
1013 //
1014 // 线程到 4×4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1015 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1016 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1017 //
1018 // 加载映射 (每线程 4 个元素, float4):
1019 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8×4=32 列 ✓
1020 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32×4=128 列 ✓
1021 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1022 //
1023 // △ Bank Conflict 提示 (面试加分点):
1024 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1025 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1026 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1027 //
1028 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1029 // Block: (32, 32, 1), 1024 线程
1030 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1031 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1032 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1033     constexpr int BM = 128;
1034     constexpr int BN = 128;
1035     constexpr int BK = 32;
1036     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1037     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1038
1039     int bx = blockIdx.x;
1040     int by = blockIdx.y;
1041     int tx = threadIdx.x;
1042     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1043
1044     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1045     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1046     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1047     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1048     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1049     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1050
1051     int load_gmem_a_m = by * BM + load_smem_a_m;
1052     int load_gmem_b_n = bx * BN + load_smem_b_n;
1053
1054     // 4×4 Thread Tile 基址 (独立于加载映射, 避免 /4 漏乘 bug)
1055     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1056     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1057
1058     float sum[4][4] = {0.f};
1059     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1060         int load_gmem_a_k = bk * BK + load_smem_a_k;
1061         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1062         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
1063         int load_gmem_b_k = bk * BK + load_smem_b_k;
1064         int load_gmem_b_addr = load_gmem_b_n * N + load_gmem_b_k;
1065         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);

```

```

1066     __syncthreads();
1067
1068 #pragma unroll
1069     for (int k = 0; k < BK; ++k) {
1070         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1071         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1072                             s_a[comp_smem_a_m_base + 1][k],
1073                             s_a[comp_smem_a_m_base + 2][k],
1074                             s_a[comp_smem_a_m_base + 3][k]};
1075         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1076                             s_b[k][comp_smem_b_n_base + 1],
1077                             s_b[k][comp_smem_b_n_base + 2],
1078                             s_b[k][comp_smem_b_n_base + 3]};
1079 #pragma unroll
1080         for (int i = 0; i < 4; ++i) {
1081             #pragma unroll
1082             for (int j = 0; j < 4; ++j) {
1083                 sum[i][j] += a_vals[i] * b_vals[j];
1084             }
1085         }
1086     }
1087     __syncthreads();
1088 }
1089
1090 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1091 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1092 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1093 #pragma unroll
1094 for (int i = 0; i < 4; ++i) {
1095     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1096     float4 reg_c;
1097     reg_c.x = sum[i][0];
1098     reg_c.y = sum[i][1];
1099     reg_c.z = sum[i][2];
1100     reg_c.w = sum[i][3];
1101     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1102 }
1103 }
1104
1105 // =====
1106 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1107 // =====
1108 // 面试要点:
1109 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1110 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1111 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1112 //   寄存器)
1113 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1114 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1115 //
1116 // ★ TN 布局详解 (面试高频考点):
1117 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1118 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1119 //   N = Normal (列优先, BLAS 原生格式)
1120 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1121 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1122 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1123 //   BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1124 //   视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1125 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1126 //   T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1127 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1128 //

```

```

1129 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1130 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1131 // N = col-major (列优先, BLAS native = normal)
1132 //
1133 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[M×N] = A[M×K] × B[K×N]
1134 // - A: row-major [M, K], B: row-major [K, N]
1135 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1136 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1137 //
1138 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1139 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1140 // - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1141 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1142 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1143 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1144 //
1145 // WGMMMA (Warp Group MMA, Hopper+):
1146 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1147 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1148 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1149 // threads) 做计算
1150 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1151 //
1152 // =====
1153 // Phase 7b-1: MMA PTX 宏定义
1154 // =====
1155 //
1156 // ---- gmem → smem: cp.async ----
1157 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3):
1158 // - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread)。
1159 // - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N)。
1160 // N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1161 // - wait_all: 等价于 commit_group + wait_group 0。
1162 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1163 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1164 #define CP_ASYNC_WAIT_GROUP(n) \
1165     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1166 //
1167 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1168 #define CP_ASYNC_CG(dst, src, bytes) \
1169     asm volatile( \
1170         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1171         "l"(src), "n"(bytes))
1172 //
1173 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1174 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1175 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1176 // aligned: 要求 128-bit 对齐, trans: 转置加载
1177 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1178     asm volatile( \
1179         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1180         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1181         : "r"(addr))
1182 //
1183 #define LDMATRIX_X2(R0, R1, addr) \
1184     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1185         : "=r"(R0), "=r"(R1) \
1186         : "r"(addr))
1187 //
1188 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1189 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1190 #define LDMATRIX_X2_T(R0, R1, addr) \
1191     asm volatile(

```

```

1192     "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n"      \
1193     : "=r"(R0), "=r"(R1)                                                  \
1194     : "r"(addr))                                                          \
1195
1196 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16 ----
1197 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1198 // row.col: A 是 row-major, B 是 col-major
1199 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1200 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1201 // C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1202 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1)      \
1203     asm volatile(                                                         \
1204         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1205         "%4, %5}, {%6, %7}, {%8, %9};\n"                                  \
1206         : "=r"(RD0), "=r"(RD1)                                           \
1207         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1208         "r"(RC1))                                                         \
1209
1210 // ---- MMA 辅助函数 ----
1211 // div_ceil: 整数除法向上取整
1212 #define HOST_DEVICE_INLINE __device__ __host__ inline
1213 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1214     return (a % b != 0) ? (a / b + 1) : (a / b);
1215 }
1216
1217 // =====
1218 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1219 // =====
1220 // 面试重点 — Tile Hierarchy:
1221 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1222 //   MMA Tile (more warps): 2×4=8 个 MMA atom → [2×16, 8×4]=[32,32]
1223 //   VAL Tile (more values): 4×4=16 expand → [32×4, 32×4]=[128,128]
1224 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1225 //           → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
1226 //
1227 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1228 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1229 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1230 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1231 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1232 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1233 //
1234 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1235 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1236 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1237 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1238 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1239 //
1240 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1241 // Block: (256, 1, 1), 8 warps
1242 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1243 // =====
1244 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1245           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1246           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1247           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1248 __global__ void __launch_bounds__(256)
1249     hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1250     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1251     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1252     const int by = blockIdx.y;
1253     constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1254     constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128

```

```

1255 constexpr int BK = MMA_K; // 16
1256
1257 // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1258 // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1259 // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
1260 // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1261 // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1262 extern __shared__ half smem[];
1263 half *s_a = smem;
1264 half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1265 constexpr int s_a_stage_offset = BM * BK; // 128*16
1266 constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)×BK(16) = B^T row-major
1267 const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1268 const int warp_id = tid / WARP_SIZE; // 0~7
1269 const int lane_id = tid % WARP_SIZE; // 0~31
1270 const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1271 const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1272
1273 // 线程到 global memory 的映射 (用于加载 A 和 B)
1274 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1275 int load_smem_a_m = tid / 2; // 0~127
1276 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1277 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1278 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1279 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1280 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1281 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1282     return;
1283
1284 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
1285 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1286 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是VAL Tile的作用。
1287 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1288
1289 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1290 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1291 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1292
1293 // 预加载前 (K_STAGE-1) 个 stage
1294 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1295 #pragma unroll
1296 for (int k = 0; k < (K_STAGE - 1); ++k) {
1297     int load_gmem_a_k = k * BK + load_smem_a_k;
1298     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1299     int load_gmem_b_k = k * BK + load_smem_b_k;
1300     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1301     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1302
1303     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1304         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1305     );
1306     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1307
1308     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1309         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1310     );
1311     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1312
1313     CP_ASYNC_COMMIT_GROUP();
1314 }
1315
1316 const int NUM_K_TILES = div_ceil(K, BK);
1317 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成

```



```

1318 __syncthreads();
1319
1320 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1321 #pragma unroll
1322 for (int k = 0; k < NUM_K_TILES; ++k) {
1323     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1324     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1325
1326     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1327     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k (row-major, 内维连续的是 K)
1328     if (k + K_STAGE - 1 < NUM_K_TILES) {
1329         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1330         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1331         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1332         // B^T: row-major [n][k], 内维连续的是 K
1333         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1334
1335         uint32_t load_smem_a_ptr =
1336             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1337                 load_smem_a_m * BK + load_smem_a_k) *
1338                 sizeof(half));
1339         CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1340
1341         uint32_t load_smem_b_ptr =
1342             (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1343                 load_smem_b_n * BK + load_smem_b_k) *
1344                 sizeof(half));
1345         CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1346         CP_ASYNC.COMMIT_GROUP();
1347     }
1348
1349     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1350     // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1351     // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1352     uint32_t RA[VAL_TILE_M][4];
1353     uint32_t RB[VAL_TILE_N][2];
1354
1355     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1356     #pragma unroll
1357     for (int i = 0; i < VAL_TILE_M; ++i) {
1358         // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1359         int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1360         // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1361         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1362         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1363         uint32_t lane_smem_a_ptr =
1364             (smem_a_base_ptr +
1365                 (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1366                 sizeof(half));
1367         LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1368     }
1369
1370     // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1371     // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1372     // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1373     #pragma unroll
1374     for (int j = 0; j < VAL_TILE_N; ++j) {
1375         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1376         int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1377         // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1378         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1379         int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1380         uint32_t lane_smem_b_ptr =

```

```

1381         (smem_b_base_ptr +
1382         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1383         sizeof(half));
1384     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1385 }
1386
1387 // MMA compute: 发射 VAL_TILE_M × VAL_TILE_N 条 MMA 指令
1388 #pragma unroll
1389 for (int i = 0; i < VAL_TILE_M; ++i) {
1390     #pragma unroll
1391     for (int j = 0; j < VAL_TILE_N; ++j) {
1392         MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1393                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1394                 RB[j][0], RB[j][1], // B fragment
1395                 RC[i][j][0], RC[i][j][1]);
1396     }
1397 }
1398
1399 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1400 if (k + K_STAGE - 1 < NUM_K_TILES) {
1401     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1402 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1403     CP_ASYNC_WAIT_GROUP(0);
1404 }
1405 __syncthreads();
1406 }
1407
1408 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1409 {
1410     for (int i = 0; i < VAL_TILE_M; ++i) {
1411         uint32_t RC0[VAL_TILE_N][4]; // 32 bits × 4 = 128 bits = 8 half
1412         uint32_t RC1[VAL_TILE_N][4]; // 32 bits × 4 = 128 bits = 8 half
1413         // =====
1414         // MMA m16n8k16 C fragment — a single 16×8 matrix (registers per warp):
1415         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1416         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1417         //
1418         //      cols: c0-1    c2-3    c4-5    c6-7
1419         //  r0:  T0{c0,c1}  T1      T2      T3      |
1420         //  r1:  T4      T5      T6      T7      |
1421         //  r2:  T8      →  T9  →  T10  →  T11    | RC[0]
1422         //  ...          |
1423         //  r7:  T28      T29      T30      T31    |
1424         //  r8:  T0{c2,c3}  T1      T2      T3      |
1425         //  r9:  T4      T5      T6      T7      |
1426         //  r10: T8      →  T9  →  T10  →  T11    | RC[1]
1427         //  ...          |
1428         //  r15: T28      T29      T30      T31    |
1429         //
1430         // Within a 4-lane group (e.g. T0-T3 for row 0):
1431         //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1432         //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1433         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1434         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1435         // =====
1436     #pragma unroll
1437     for (int j = 0; j < VAL_TILE_N; ++j) {
1438         RC0[j][0] = RC[i][j][0];
1439         RC1[j][0] = RC[i][j][1];
1440         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1441         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1442         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1443         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);

```



```

1444     RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1445     RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1446 }
1447 // 每 4 个 lane 中只有 lane 0 做 128-bit store
1448 // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行) :
1449 //
1450 // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1451 // |-----|-----|-----|-----|
1452 // | 0~3    | 0        | row 0     | row 8     |
1453 // | 4~7    | 1        | row 1     | row 9     |
1454 // | 8~11   | 2        | row 2     | row 10    |
1455 // | 12~15  | 3        | row 3     | row 11    |
1456 // | 16~19  | 4        | row 4     | row 12    |
1457 // | 20~23  | 5        | row 5     | row 13    |
1458 // | 24~27  | 6        | row 6     | row 14    |
1459 // | 28~31  | 7        | row 7     | row 15    |
1460 //
1461 if (lane_id % 4 == 0) {
1462     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1463     int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1464     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1465 #pragma unroll
1466     for (int j = 0; j < VAL_TILE_N; ++j) {
1467         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1468         int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1469         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1470         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1471         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1472         // 128-bit store: 一次写入 8 个 half
1473         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1474             *reinterpret_cast<float4*>(&RC0[j][0]);
1475         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1476             *reinterpret_cast<float4*>(&RC1[j][0]);
1477     }
1478 }
1479 }
1480 }
1481 }
1482
1483 // =====
1484 // Phase 7c: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
1485 // =====
1486 // 面试要点 (WGMMMA vs MMA 对比) :
1487 // - MMA: warp 级 (32 threads) , 同步执行
1488 // - WGMMMA: warpgroup 级 (128 threads = 4 warps) , 异步执行 (fire-and-forget)
1489 // - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4x16=64倍)
1490 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
1491 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
1492 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
1493
1494 // ---- WGMMMA 辅助函数 ----
1495 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
1496 // - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
1497 //   (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行) 。
1498 // - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N) 。
1499 //   N=0 → 等所有 group 完成。* 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3) ,
1500 //   都是 "wait until only N or fewer groups are pending"。
1501 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
1502 #define WGMMMA_COMMIT_GROUP() \
1503     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
1504 #define WGMMMA_WAIT_GROUP(n) \
1505     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
1506

```

```

1507 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
1508 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
1509 //   encoded = (x & 0x3FFFF) >> 4
1510 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
1511 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
1512 // 可省略以扩大可表示范围。
1513 // 解码: decoded = encoded << 4 (回到原始 byte 值) 。
1514 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4)
1515
1516 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
1517 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
1518 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
1519 //
1520 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
1521 // -----
1522 // | 位域           | 范围       | 大小 | 含义
1523 // |-----|-----|-----|-----
1524 // | start_address   | [0, 14)    | 14   | smem 基址编码
1525 // | (unused)        | [14, 16)   | 2    | -
1526 // | leading_byte_offset | [16, 30)   | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
1527 // | (unused)        | [30, 32)   | 2    | -
1528 // | stride_byte_offset | [32, 46)   | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
1529 // | (unused)        | [46, 49)   | 3    | -
1530 // | base_offset     | [49, 52)   | 3    | swizzle pattern 偏移
1531 // | (unused)        | [52, 62)   | 10   | -
1532 // | layout_type     | [62, 64)   | 2    | swizzle 模式
1533 // -----
1534 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
1535 //
1536 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
1537 // - 128B swizzle atom 在 128-bit 单位下为 8x8
1538 // - 归一化到 half(2B) 后: atom 覆盖 8x(128/16)=64 个 K 方向元素 x 8 个 M 方向元素
1539 // - 即每个 atom = 64(K) x 8(M) 个 half = 64x8x2 = 1024 bytes
1540 // - 这是 stride_byte_offset=1024 的来源
1541 //
1542 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
1543 //   此处填 16 仅为占位, 编码后为 1。
1544 //
1545 // - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。
1546 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
1547 //
1548 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
1549 _device_ inline uint64_t make_smem_desc(half *ptr) {
1550 // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
1551 // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址) 。
1552 uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
1553
1554 // 从全零开始: 所有位域初始化为 0。
1555 uint64_t desc = 0x0000000000000000;
1556
1557 // — bits [0, 14): start_address —
1558 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
1559 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
1560 // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
1561 desc |= SMEM_DESC_ENCODE(addr);
1562
1563 // — bits [16, 30): leading_byte_offset —
1564 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
1565 // << 16 将编码值放到 bits [16, 30)。
1566 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
1567 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
1568 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
1569 // 对 INTERLEAVE: 同一 8x2 brick 内第一列到第二列的字节偏移。

```

```

1570 // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
1571 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
1572
1573 // — bits [32, 46): stride_byte_offset —
1574 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
1575 // << 32 将编码值放到 bits [32, 46)。
1576 // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2)。
1577 // 1024 的来源 (K-Major + 128B swizzle + half) :
1578 // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
1579 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
1580 // 若为 MN-Major: SB0 是从首列到下一列的偏移 (8 列一组)。
1581 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
1582
1583 // — bits [62, 64): layout_type (swizzle mode) —
1584 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
1585 // 1 = 128B swizzle mode。
1586 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
1587 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
1588 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
1589 desc |= 1llu << 62;
1590
1591 return desc;
1592 }
1593
1594 // ---- WGMMA PTX 指令宏 ----
1595 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
1596 // m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算 D[64×128] += A[64×16] × B[16×128]。
1597 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
1598 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
1599 // 每线程 64 half = 32 uint32。
1600 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
1601 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
1602 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
1603 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
1604 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
1605 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
1606                                     TransB) \
1607 { \
1608     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
1609     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
1610     asm volatile( \
1611         "{\n" \
1612         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
1613         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
1614         "%8, %9, %10, %11, %12, %13, %14, %15, " \
1615         "%16, %17, %18, %19, %20, %21, %22, %23, " \
1616         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
1617         "%32, " \
1618         "%33, " \
1619         "%34, %35, %36, %37, %38;\n" \
1620         "}\n" \
1621         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
1622           "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
1623           "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
1624           "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
1625           "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
1626           "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
1627           "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
1628           "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
1629         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
1630           "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
1631           "n"(int32_t(TransB))); \
1632 }

```

```

1633
1634 // ---- TMA Shared Memory Layout ----
1635 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
1636 //
1637 // 每个 stage 包含两块 smem:
1638 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
1639 //   B tile: [BK, BN] row-major → BN×BK 个 half, 地址连续
1640 //
1641 // ★ 关键区别 — WGMMMA 的 B 布局 vs MMA(TN) 的 B^T 布局:
1642 //   MMA (TN):   smem 存 B^T [N, K] row-major, ldmatrix 解出 col-major B fragment。
1643 //   WGMMMA:     smem 存 B [BK, BN] row-major (即 N 维连续, K 维步幅=BN个half)。
1644 //               WGMMMA 指令的 imm-trans-b=0 指示硬件按 K-major 读 B,
1645 //               128B swizzle + make_smem_desc 的 stride_byte_offset 将这些地址
1646 //               重新映射, 使硬件能从 [BK,BN] row-major 中正确按 K-major 顺序取值。
1647 //   简言之: swizzle 桥接了 "row-major 存储" 和 "K-major 读取" 的差异。
1648 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
1649     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
1650     // B: K-major 布局, 供 WGMMMA imm-trans-b=0 直接读取, [BK, BN].
1651     alignas(128) half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN]
1652 };
1653
1654 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
1655 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
1656 //
1657 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
1658 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
1659 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
1660 //
1661 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
1662 //   将 A/B tile 从 HBM 搬到 shared memory。
1663 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
1664 //
1665 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
1666 //   - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
1667 //   - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖
1668 //
1669 // 本节重点理解:
1670 //   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
1671 //       即可搬运整个 2D tile, 无需所有线程参与。
1672 //   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
1673 //   3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
1674 //       Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
1675 //
1676 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
1677 //   WGMMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
1678 //   K Tile:           BK=64, 每个 K tile 包含 BK/WGMMMA_K=4 个 WGMMMA atom
1679 //   M Tile:           BM=128, 每个 M tile 包含 BM/WGMMMA_M=2 个 WGMMMA atom
1680 //   N Tile:           BN=128, 单个 WGMMMA_N=128 即可覆盖, 无需在 N 方向分块
1681 //   Block Tile:       C[128,128] = A[128,64] × B[64,128]
1682 //   Threads:          256 = 2 warpgroups × 128 threads/warpgroup
1683 //   Producer(WG0):    128 threads (仅 thread 0 做 TMA 提交)
1684 //   Consumer(WG1):    128 threads = 4 warps (全部参与 WGMMMA 和写回)
1685 //
1686 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
1687 // 的延迟被计算完全掩盖。
1688 //
1689 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1690 //   - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
1691 // Block: (256, 1, 1), 2 warpgroups
1692 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
1693 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
1694         const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
1695         const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,

```

```

1696         const bool BLOCK_SWIZZLE = false>
1697 __global__ void __launch_bounds__(NUM_THREADS)
1698     hgemm_wgmma_stages_tn(
1699         int M, int N, int K, half *C,
1700         const CUtensorMap *__restrict__ tensorMapA,
1701         const CUtensorMap *__restrict__ tensorMapB) {
1702     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
1703     // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
1704     // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
1705     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
1706     // 不展开宿主侧 create_tensor_map 细节。
1707
1708     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1709     // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
1710     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1711     const int by = blockIdx.y;
1712     // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
1713     // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
1714     constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
1715     // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
1716     // 当只有一个 consumer 时, 它负责全部 BM=128 行
1717     constexpr int B_WG_M = BM / num_consumers; // 128
1718
1719     // 边界检查: 确保当前 block 不超出 M/N 范围
1720     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
1721         return;
1722
1723     // ---- Shared Memory 分配 ----
1724     // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
1725     // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
1726     extern __shared__ __align__(128) uint8_t smem_AB[];
1727     WgmmaSMem<BM, BN, BK, K_STAGE> &s =
1728         *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE>*>(smem_AB);
1729     half *s_a = s.A;
1730     half *s_b = s.B;
1731
1732     // ---- cuda::barrier 初始化 ----
1733     //
1734     // ★ Barrier 机制速查 (arrive / wait 核心语义):
1735     //
1736     // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
1737     //
1738     //   init(bar, N): 设置 arrive_count = N。
1739     //       含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
1740     //
1741     //   arrive(): 线程声明"我已到达此 barrier 点"。
1742     //       - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
1743     //       - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
1744     //
1745     //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
1746     //       - 即: 等待 phase 翻转。
1747     //       - Phase 翻转条件: (1) arrive 次数达到 arrive_count
1748     //                       (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
1749     //
1750     //   典型用法: b.wait(b.arrive())
1751     //       - arrive() 注册自己到达, 拿到 token (当前 phase = P)
1752     //       - wait(token) 阻塞直到 phase 翻转 (P → P+1)
1753     //       - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
1754     //       - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
1755     //
1756     // ★ 本 kernel 的 Pipeline 同步协议 (K_STAGE=3, arrive_count=129):
1757     //
1758     //   Producer (1 thread)                Consumer (128 threads)

```

```

1759 //
1760 // C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
1761 // ┌ for each k_tile: ┐ ┌ for each k_tile: ┐
1762 // | P1: arrive+wait(empty[q]) | C1: arrive+wait(full[q])
1763 // |   ↓ 等 stage q 被消费完 |   ↓ 等 TMA 数据就绪
1764 // | P2: TMA(A[q]) + TMA(B[q]) | C2: WGMMMA 计算
1765 // |   ↓ 异步拷贝 | C3: wait WGMMMA 完成
1766 // | P3: arrive_tx(full[q]) | C4: arrive(empty[q])
1767 // |   ↑ 通知: stage q 数据就绪 |   ↑ 通知: stage q 已消费完
1768 // └──────────────────┘ └──────────────────┘
1769 //
1770 // 关键不变式 (invariant) :
1771 //   - Producer 不会覆盖 Consumer 正在读的 stage
1772 //   - Consumer 不会读 Producer 还没写完的 stage
1773 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
1774 //
1775 // 每个 stage 有两个 barrier:
1776 //   full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx) , Consumer 等 (wait) 。
1777 //   empty[qidx]: Stage 空闲信号。 Consumer 发 (arrive) , Producer 等 (wait) 。
1778 //
1779 // arrive_count = 128 (consumer) + 1 (producer) = 129:
1780 // 每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
1781 #pragma nv_diag_suppress static_var_with_dynamic_init
1782 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
1783 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
1784 #pragma nv_diag_default static_var_with_dynamic_init
1785
1786 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
1787 const int num_blocks_k = K / BK;
1788 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
1789 const int tid = threadIdx.x % 128; // 0~127 within warpgroup
1790
1791 // 初始化 barriers (仅 thread 0 执行)
1792 if (threadIdx.x == 0) {
1793     for (int i = 0; i < K_STAGE; ++i) {
1794         // arrive_count = 129: 128 consumer + 1 producer
1795         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内) ,
1796         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
1797         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
1798         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
1799         init(&empty[i], num_consumers * 128 + 1); // same
1800     }
1801     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
1802     // 对 async proxy (TMA/WGMMMA) 可见。
1803     cuda::device::experimental::fence_proxy_async_shared_cta();
1804 }
1805 __syncthreads();
1806
1807 // =====
1808 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
1809 // =====
1810 //
1811 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
1812 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。
1813 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
1814 //
1815 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
1816 //   Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
1817 //   Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
1818 //
1819 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
1820 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
1821 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完

```



```

1822 //
1823 // 这是生产者-消费者模型的 GPU 实现：不需要共享循环变量，barrier 的 phase
1824 // 交替天然保证了流水线串行化 (at most K_STAGE-1 steps ahead)。
1825 // =====
1826 // Producer Warpgroup (WG0, threadIdx.x 0~127)
1827 // 职责：提交 TMA 2D 拷贝，将 A/B tile 从 HBM 异步搬运到 SMEM。
1828 // 只有 tid==0 执行实际拷贝提交，其余 127 个线程空闲。
1829 // =====
1830 if (wg_idx == 0) {
1831     if (tid == 0) {
1832         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
1833         int qidx = 0;
1834         for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1835             ++block_k_iter, ++qidx) {
1836             if (qidx == K_STAGE)
1837                 qidx = 0;
1838
1839             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
1840             //
1841             // 模式 empty[qidx].wait(empty[qidx].arrive()):
1842             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达，
1843             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
1844             //     128 次 arrive (来自上一轮迭代或 C0 初始化)，则加上这次共 129 次
1845             //     → phase 立即翻转。
1846             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
1847             //     由于 arrive() 是第 129 次到达，phase 在 arrive 时已翻转，
1848             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
1849             //
1850             // 语义：Producer 说"我准备好写 stage qidx 了，Consumer 用完没有？"
1851             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
1852             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
1853             empty[qidx].wait(empty[qidx].arrive());
1854
1855             // Step P2: 提交 TMA 2D 拷贝指令
1856             // cp_async_bulk_tensor_2d_global_to_shared:
1857             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
1858             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
1859             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
1860             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
1861             //
1862             // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile，
1863             // 无需线程逐元素搬运，零寄存器开销。
1864             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
1865             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
1866                 &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
1867                 full[qidx]);
1868
1869             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
1870             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
1871                 &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
1872                 full[qidx]);
1873
1874             // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
1875             //
1876             // barrier_arrive_tx(bar, arrive_count_update, byte_count):
1877             //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
1878             //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
1879             //   - Phase 翻转条件：
1880             //     (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
1881             //     (b) 所有声明的 async 字节已写入 smem
1882             //   两个条件都满足后 phase 才翻转，Consumer 的 wait() 才返回。
1883             //   - 即：Consumer 不会在 TMA 数据完整到达前就开始读 smem。
1884             [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(

```



```

1885         full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
1886     }
1887 }
1888 }
1889 // =====
1890 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
1891 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
1892 // 所有 128 个线程 (4 warps) 全部参与。
1893 // =====
1894 else {
1895     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
1896     //
1897     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
1898     // 这是 Pipeline 的"起搏"步骤——没有它, Producer 的 empty[qidx].wait()
1899     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
1900     //
1901     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
1902     // Producer 后续的 empty[qidx].arrive() 作为第 129 次, 触发 phase 翻转。
1903     for (int i = 0; i < K_STAGE; ++i) {
1904         [[maybe_unused]] auto token = empty[i].arrive();
1905     }
1906
1907     // 累加器寄存器声明
1908     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
1909     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
1910     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
1911     //   - d[*][g][*]: N 方向第 g 组 16 列
1912     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
1913     // 每个线程总共 2 * 8 * 4 = 64 uint32 = 128 half
1914     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
1915     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
1916
1917     int qidx = 0;
1918     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
1919     for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1920          ++block_k_iter, ++qidx) {
1921         if (qidx == K_STAGE)
1922             qidx = 0;
1923
1924         // Step C1: 等待 TMA 数据就绪 (full 信号)
1925         //
1926         // 模式 full[qidx].wait(full[qidx].arrive()):
1927         //   - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
1928         //   - 当前 phase 累积: 128/129, phase 尚未翻转。
1929         //   - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
1930         //     贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
1931         //
1932         // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有?"
1933         full[qidx].wait(full[qidx].arrive());
1934
1935         // Step C2: 发射 WGMMMA 指令序列
1936         //
1937         // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
1938         // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
1939         //
1940         // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
1941         //   - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
1942         //   - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
1943         //   - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
1944         //
1945         // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
1946         //   D[64x128] += A[64x16] × B[16x128]
1947         //   A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。

```

```

1948 // ScaleD=1: 累加。K 维有 BK/WGMMMA_K=4 次迭代，每次都 ScaleD=1。
1949 WGMMMA_FENCE();
1950
1951 // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
1952 // 每个 atom 处理 64 行 M 方向数据。
1953 #pragma unroll
1954 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1955     // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
1956     // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
1957     half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
1958
1959     // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)
1960     // 每次处理 K=16 维的矩阵乘，4 次累加后覆盖完整的 BK=64。
1961     #pragma unroll
1962     for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
1963         // 第 k_it 次 WGMMMA:
1964         // A: wgmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
1965         // B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
1966         // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
1967         // 第 k_it*16 行开始取 16 行，每行 BN=128 列。
1968         // ScaleD=1: 累加 (K 维迭代需要累积结果)
1969         WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
1970                                     s_b + qidx * BK * BN + k_it * WGMMMA_K,
1971                                     1, // ScaleD=1: accumulate (不清零)
1972                                     1, 1, 0, 0);
1973     }
1974 }
1975
1976 // Step C3: 提交并等待 WGMMMA 完成
1977 //
1978 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
1979 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组，提交到 async proxy 执行。
1980 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
1981 //
1982 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
1983 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
1984 // * 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
1985 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
1986 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
1987 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
1988 WGMMMA_COMMIT_GROUP();
1989 WGMMMA_WAIT_GROUP(0);
1990
1991 // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
1992 //
1993 // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
1994 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
1995 //
1996 // 语义: Consumer 说 "stage qidx 我已经读完了, 你可以放心覆盖了。"
1997 [[maybe_unused]] auto token = empty[qidx].arrive();
1998 }
1999
2000 // =====
2001 // Epilogue: 将寄存器中的累加结果写回 global memory C
2002 // =====
2003 //
2004 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
2005 //
2006 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
2007 // 这些 half 分布在 d[2][8][4] 数组中:
2008 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
2009 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
2010 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half

```

```

2012 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
2013 //
2014 // 其中:
2015 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
2016 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
2017 //
2018 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
2019 // +-----+-----+
2020 // | reg[0] | reg[2] | <- rows [row, row+8)
2021 // |(col,+2)|(col+8)|
2022 // +-----+-----+
2023 // | reg[1] | reg[3] | <- rows [row+8, row+16)
2024 // |(col,+2)|(col+8)|
2025 // +-----+-----+
2026 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
2027 //
2028 // 三维遍历:
2029 // m_it=0: rows 0~63, m_it=1: rows 64~127
2030 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
2031 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
2032 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
2033
2034 const int lane = tid % 32;
2035 const int warp = tid / 32;
2036 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
2037 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
2038 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
2039 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
2040 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
2041 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
2042 const int row = warp * 16 + lane / 4;
2043 // block_C: 当前 C tile 在 global memory 中的起始地址
2044 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
2045 half *block_C = C + by * BM * N + bx * BN;
2046
2047 #pragma unroll
2048 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2049     int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
2050 #pragma unroll
2051 for (int g = 0; g < WGMMMA_N / 16; ++g) {
2052     int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
2053     // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
2054     // 左上象限: (row, col)
2055     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
2056         d[m_it][g][0];
2057     // 左下象限: (row+8, col)
2058     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
2059         d[m_it][g][1];
2060     // 右上象限: (row, col+8)
2061     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
2062         d[m_it][g][2];
2063     // 右下象限: (row+8, col+8)
2064     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
2065         d[m_it][g][3];
2066 }
2067 }
2068 }
2069
2070 #if (defined(NOTES_V2_HAS_WGMMMA) \
2071     || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) \
2072     || defined(__CUDA_ARCH_FEAT_SM90_ALL))
2073 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----

```

```

2074 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled) :
2075 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
2076 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
2077 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
2078 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
2079 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
2080 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
2081 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
2082 //
2083 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
2084
2085 template <int BlockMajorSize, int BlockMinorSize>
2086 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
2087                                               half *gmem_ptr,
2088                                               int blocks_height,
2089                                               int blocks_width) {
2090     void *gmem_address = (void *)gmem_ptr;
2091     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
2092                                     (uint64_t)BlockMajorSize * blocks_height,
2093                                     1, 1, 1};
2094     uint64_t gmem_prob_stride[5] = {
2095         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
2096     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
2097                                    uint32_t(BlockMajorSize), 1, 1, 1};
2098     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
2099     CUresult result = cuTensorMapEncodeTiled(
2100         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
2101         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
2102         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
2103         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
2104     if (result != CUDA_SUCCESS)
2105         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
2106 }
2107
2108 __host__ static inline CUtensorMap *allocate_and_create_tensor_map(
2109     half *src, int blocks_height, int blocks_width) {
2110     CUtensorMap *tma_map_d;
2111     cudaMalloc(&tma_map_d, sizeof(CUtensorMap));
2112     CUtensorMap tma_map_host;
2113     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
2114     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUtensorMap),
2115               cudaMemcpyHostToDevice);
2116     return tma_map_d;
2117 }
2118 #endif /* NOTES_V2_HAS_WGMMMA */
2119
2120 // =====
2121 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
2122 // =====
2123 // 面试要点 (FlashAttention 算法) :
2124 //   1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
2125 //     但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
2126 //   2. FA 三板斧:
2127 //     a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
2128 //     b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
2129 //     c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
2130 //   3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
2131 //     - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
2132 //     - 优点: 减少 warp 间通信和 shuffle
2133 //   4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
2134 //     for each K,V tile:
2135 //         S_cur = Q @ K^T // 未缩放, 存入 R_S
2136 //         m_new = max(m_old, row_max(S_cur * scale))

```

```

2137 //      P_cur = exp(S_cur * scale - m_new)      // ← 写回 R_S 寄存器!
2138 //      l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
2139 //      O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
2140 //      O_final = O_new / l_final
2141 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
2142 //      - R_S 经过 softmax 后, 存储的是 P = exp(S - m), 数据仍然是 half 精度
2143 //      - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
2144 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
2145 //      - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
2146 //      都天然同构”的通用结论
2147 //
2148 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
2149 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
2150 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
2151 // Block: (128, 1, 1), kNumThreads=WARP_SIZE*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
2152 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
2153
2154 // ---- 寄存器填充辅助函数 ----
2155 template <typename T, int M, const int N, const int K = 2>
2156 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
2157     #pragma unroll
2158     for (int i = 0; i < M; ++i)
2159     #pragma unroll
2160         for (int j = 0; j < N; ++j)
2161         #pragma unroll
2162             for (int k = 0; k < K; ++k)
2163                 R[i][j][k] = val;
2164 }
2165
2166 template <typename T, int M, const int N = 2>
2167 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
2168     #pragma unroll
2169     for (int i = 0; i < M; ++i)
2170     #pragma unroll
2171         for (int j = 0; j < N; ++j)
2172             R[i][j] = val;
2173 }
2174
2175 // =====
2176 // FlashAttention-2 Split-Q Kernel (完整实现)
2177 // =====
2178 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
2179 //
2180 // Tile 设计 (以 kHeadDim=64 为例) :
2181 //      Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
2182 //      Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
2183 //      Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
2184 //
2185 // 执行流程:
2186 //      1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
2187 //      2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
2188 //      3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
2189 //      4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
2190 //      5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
2191 //           → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
2192 //      6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
2193 //           → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
2194 //      7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
2195 //      8) 最终 rescale: O_final = (1/l_final) * O_final
2196 //      9) Epilogue: warp shuffle + 128-bit collective store
2197
2198 template <
2199     const int kHeadDim,                // head dim: 32, 64, 128

```

```

2200 const int kMmaAtomM,          // 16 (MMA instruction M dimension)
2201 const int kMmaAtomN,          // 8  (MMA instruction N dimension)
2202 const int kMmaAtomK,          // 16 (MMA instruction K dimension)
2203 const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
2204 const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
2205 const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
2206 const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
2207 const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
2208 const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
2209 const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
2210 const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
2211 const int kStage,             // pipeline stages for K: 1 or 2
2212 const int kPad>              // padding for bank conflict avoidance
2213 __global__ void __launch_bounds__(WARP_SIZE *kMmaTileSeqLenQ *kMmaTileSeqLenK)
2214     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
2215                                     int QKV_seqlen, int QKV_head) {
2216     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
2217     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
2218     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
2219     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
2220     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
2221     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
2222     const float scale = 1.0f / sqrtf((float)kHeadDim);
2223
2224     // Block indexing
2225     const int QKV_batch_id = blockIdx.y / QKV_head;
2226     const int QKV_head_id = blockIdx.y % QKV_head;
2227     const int Q_tile_id = blockIdx.x;
2228     const int tid = threadIdx.x;
2229     const int warp_id = tid / WARP_SIZE;
2230     const int lane_id = tid % WARP_SIZE;
2231     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
2232     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
2233
2234     // Global memory base offsets for this (batch, head)
2235     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
2236     const int Q_gmem_offset =
2237         (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
2238         kHeadDim;
2239     const int K_gmem_offset = Q_gmem_offset;
2240     const int V_gmem_offset = Q_gmem_offset;
2241     const int O_gmem_offset = Q_gmem_offset;
2242
2243     // Thread-to-smem mapping for cooperative load
2244     int load_smem_Q_Br = tid / (kNumThreads / Br);
2245     int load_smem_Q_d =
2246         (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
2247     int load_smem_K_Bc = tid / (kNumThreads / Bc);
2248     int load_smem_K_d =
2249         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2250     int load_smem_V_Bc = tid / (kNumThreads / Bc);
2251     int load_smem_V_d =
2252         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2253
2254     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
2255     if (load_gmem_Q_Br >= QKV_seqlen)
2256         return;
2257
2258     // ---- Shared memory layout ----
2259     extern __shared__ half smem[];
2260     constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
2261     constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
2262     half *Q_tile_smem = smem;

```



```

2263 half *K_tile_smem = Q_tile_smem + Q_tile_size;
2264 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
2265 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
2266 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
2267
2268 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
2269 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
2270 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
2271
2272 // ---- Online Softmax persistent state ----
2273 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
2274 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
2275 float lane_block_row_max_old[kValTileSeqLenQ][2];
2276 float lane_block_row_sum_old[kValTileSeqLenQ][2];
2277 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
2278 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
2279
2280 // ---- Register allocation ----
2281 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
2282 uint32_t R_K[kValTileSeqLenK][2]; // K regs
2283 uint32_t R_V[kValTileHeadDimV][2]; // V regs
2284 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
2285 // 对单个 m16n8k16 tile 而言:
2286 // - reg[0] 持有该 tile 前 8 行里的两个 half 值
2287 // - reg[1] 持有该 tile 后 8 行里的两个 half 值
2288 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
2289 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
2290 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
2291 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
2292           [2]; // O accumulator (final output)
2293
2294 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2295 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
2296
2297 // =====
2298 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
2299 // =====
2300 {
2301     int load_gmem_Q_addr =
2302         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
2303     uint32_t load_smem_Q_ptr =
2304         smem_Q_base_ptr +
2305         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
2306 #pragma unroll
2307     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
2308         CP_ASYNC.CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
2309     }
2310     CP_ASYNC.COMMIT_GROUP();
2311 }
2312
2313 // =====
2314 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
2315 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
2316 // 后续在外循环里不断递增至 tile 1/2/3/.../Tc-1。
2317 // =====
2318 if constexpr (kStage > 1) {
2319 #pragma unroll
2320     for (int stage = 0; stage < (kStage - 1); ++stage) {
2321         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
2322         int load_gmem_K_addr =
2323             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2324         uint32_t load_smem_K_ptr =
2325             smem_K_base_ptr +

```



```

2326         (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2327         load_smem_K_d) *
2328         sizeof(half);
2329 #pragma unroll
2330     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2331         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2332     }
2333     CP_ASYNC_COMMIT_GROUP();
2334 }
2335 CP_ASYNC_WAIT_GROUP(kStage - 2);
2336 __syncthreads();
2337 }
2338
2339 // =====
2340 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
2341 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
2342 // =====
2343 #pragma unroll 1
2344 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
2345     int smem_sel = tile_K_seqlen % kStage;
2346     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
2347
2348     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
2349     if constexpr (kStage > 1) {
2350         // 只有 kStage>1 才能真正做 K 的 pipeline:
2351         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
2352         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
2353         // Load current V tile (no pipeline for V — one stage is enough)
2354         {
2355             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
2356             int load_gmem_V_addr =
2357                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
2358             uint32_t load_smem_V_ptr =
2359                 smem_V_base_ptr +
2360                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
2361 #pragma unroll
2362             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2363                 CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
2364             }
2365             CP_ASYNC_COMMIT_GROUP();
2366         }
2367
2368         // Prefetch next K tile (pipelined)
2369         if ((tile_K_seqlen + 1) < Tc) {
2370             int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
2371             int load_gmem_K_addr =
2372                 K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2373             uint32_t load_smem_K_ptr =
2374                 smem_K_base_ptr +
2375                 (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2376                 load_smem_K_d) *
2377                 sizeof(half);
2378 #pragma unroll
2379             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2380                 CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2381             }
2382             CP_ASYNC_COMMIT_GROUP();
2383         }
2384     }
2385
2386     // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
2387     fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2388 #pragma unroll

```

```

2389     for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
2390         // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
2391         #pragma unroll
2392         for (int i = 0; i < kValTileSeqLenQ; ++i) {
2393             int warp_smem_Q_Br =
2394                 warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
2395             int lane_smem_Q_Br =
2396                 warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
2397             int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
2398             uint32_t lane_smem_Q_ptr =
2399                 smem_Q_base_ptr +
2400                 (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
2401             LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
2402                 lane_smem_Q_ptr);
2403         }
2404
2405         // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
2406         // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
2407         #pragma unroll
2408         for (int j = 0; j < kValTileSeqLenK; ++j) {
2409             int warp_smem_K_Bc =
2410                 warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
2411             int lane_smem_K_Bc =
2412                 warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
2413             int lane_smem_K_d =
2414                 tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
2415             uint32_t lane_smem_K_ptr =
2416                 smem_K_base_ptr +
2417                 (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
2418                     lane_smem_K_d) *
2419                     sizeof(half);
2420             LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
2421         }
2422
2423         // MMA: S[tile] += Q[tile] @ K^T[tile]
2424         #pragma unroll
2425         for (int i = 0; i < kValTileSeqLenQ; ++i) {
2426             #pragma unroll
2427             for (int j = 0; j < kValTileSeqLenK; ++j) {
2428                 HMMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
2429                     R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
2430                     R_S[i][j][1]);
2431             }
2432         }
2433     } // end loop over d
2434     __syncthreads();
2435
2436     // =====
2437     // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
2438     // =====
2439     // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
2440     //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
2441     //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
2442     //   - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
2443     //   - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
2444     // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
2445     // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
2446     // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
2447     // kValTileSeqLenK tiles.
2448
2449     float lane_row_max_new[kValTileSeqLenQ][2];
2450     float lane_row_sum_new[kValTileSeqLenQ][2];
2451     fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);

```

```

2452     fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
2453
2454     // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
2455     #pragma unroll
2456     for (int i = 0; i < kValTileSeqLenQ; ++i) {
2457         #pragma unroll
2458         for (int j = 0; j < kValTileSeqLenK; ++j) {
2459             // Extract half values from R_S registers (C matrix fragment layout)
2460             float2 t_reg_S_0 =
2461                 __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
2462             float2 t_reg_S_1 =
2463                 __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
2464             // S = (Q@K^T) * scale
2465             float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
2466             float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
2467             lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
2468             lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
2469         }
2470         // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
2471         // {0,4,8,...,28} hold valid data)
2472         lane_row_max_new[i][0] =
2473             warp_reduce_max<4, float>(lane_row_max_new[i][0]);
2474         lane_row_max_new[i][1] =
2475             warp_reduce_max<4, float>(lane_row_max_new[i][1]);
2476     }
2477
2478     // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
2479     // 面试关键点: 这里将 P 写回 R_S 寄存器!
2480     // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
2481     // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
2482     #pragma unroll
2483     for (int i = 0; i < kValTileSeqLenQ; ++i) {
2484         // m_new = max(m_old, m_cur)
2485         float block_row_max_new_0 =
2486             max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
2487         float block_row_max_new_1 =
2488             max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
2489
2490         #pragma unroll
2491         for (int j = 0; j < kValTileSeqLenK; ++j) {
2492             float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
2493             float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
2494
2495             // P = exp(S * scale - m_new), 用 fma 保证精度
2496             t_reg_S_0.x =
2497                 __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
2498             t_reg_S_0.y =
2499                 __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
2500             t_reg_S_1.x =
2501                 __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
2502             t_reg_S_1.y =
2503                 __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
2504
2505             // Accumulate row sums
2506             lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
2507             lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
2508
2509             // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
2510             HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
2511             HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
2512         }
2513
2514         // Warp-level reduce sum (warp_size = 4, same as max)

```

```

2515     lane_row_sum_new[i][0] =
2516         warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
2517     lane_row_sum_new[i][1] =
2518         warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
2519 }
2520 __syncthreads();
2521
2522 // =====
2523 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
2524 // =====
2525 // Wait for V to be ready before computing P@V
2526 if constexpr (kStage > 1) {
2527     if ((tile_K_seqlen + 1) < Tc) {
2528         CP_ASYNC_WAIT_GROUP(1);
2529     } else {
2530         CP_ASYNC_WAIT_GROUP(0);
2531     }
2532 } else {
2533     CP_ASYNC_WAIT_GROUP(0);
2534 }
2535 __syncthreads();
2536
2537 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
2538
2539 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
2540 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
2541 #pragma unroll
2542 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
2543     // ldmatrix.x2.trans: load V[Bc,d] with transposition
2544     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
2545     // transposed ldmatrix
2546 #pragma unroll
2547 for (int j = 0; j < kValTileHeadDimV; ++j) {
2548     int warp_smem_V_d =
2549         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2550     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
2551     int lane_smem_V_d = warp_smem_V_d;
2552     uint32_t lane_smem_V_ptr =
2553         smem_V_base_ptr +
2554         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
2555     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
2556 }
2557
2558 // P matrix layout in R_S[i][j][2]:
2559 //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
2560 //   | 64x64 | warp_KV 0 |
2561 //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
2562 //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
2563 //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
2564 //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
2565 // tile_V_Bc selects which 16-column slice of P to use:
2566 //   tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
2567 //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
2568 //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
2569 //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
2570 // 对应的 MMA A fragment 布局可以这样记:
2571 //   - rows 0~7: lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
2572 //   - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
2573 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
2574 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
2575 // fragment 都无条件成立的通用结论。layout转换逻辑:
2576 //   C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
2577 int w = tile_V_Bc * 2;

```

```

2578 #pragma unroll
2579     for (int i = 0; i < kValTileSeqLenP; ++i) {
2580 #pragma unroll
2581         for (int j = 0; j < kValTileHeadDimV; ++j) {
2582             MMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
2583                     R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
2584                     R_V[j][0], R_V[j][1], // B fragment = V
2585                     R_0[i][j][0], R_0[i][j][1]); // C fragment output
2586         }
2587     }
2588 } // end for tile_V_Bc
2589 __syncthreads();
2590
2591 // =====
2592 // 3e: Online rescaling —  $O_{new} = \exp(m_{old} - m_{new}) * O_{old} + P@V$ 
2593 // =====
2594 // 公式来源: FA2 paper Eq.(7-8), 使用  $\exp(m_{old} - m_{new})$  做 0 与 1 的 rescale
2595 #pragma unroll
2596     for (int i = 0; i < kValTileSeqLenP; ++i) {
2597         float block_row_max_new_0 = lane_row_max_new[i][0];
2598         float block_row_max_new_1 = lane_row_max_new[i][1];
2599         float block_row_sum_new_0 = lane_row_sum_new[i][0];
2600         float block_row_sum_new_1 = lane_row_sum_new[i][1];
2601
2602         float block_row_max_old_0 = lane_block_row_max_old[i][0];
2603         float block_row_max_old_1 = lane_block_row_max_old[i][1];
2604
2605         block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
2606         block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
2607
2608         // Handle first iteration:  $m_{old} = -inf$ , need to use  $m_{new}$  directly
2609         block_row_max_old_0 =
2610             (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
2611         block_row_max_old_1 =
2612             (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
2613
2614         float rescale_o_factor_0 =
2615             __expf(block_row_max_old_0 - block_row_max_new_0);
2616         float rescale_o_factor_1 =
2617             __expf(block_row_max_old_1 - block_row_max_new_1);
2618
2619         // Rescale  $O_{old}$  + Add  $P@V$  in one fused step
2620 #pragma unroll
2621         for (int j = 0; j < kValTileHeadDimV; ++j) {
2622             //  $R_0 / R_D$  与前面的  $R_S$  一样, 都按 MMA C fragment 布局解释:
2623             //   reg[0] -> rows 0~7 的 {c0,c1}
2624             //   reg[1] -> rows 8~15 的 {c2,c3}
2625             float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
2626             float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
2627             float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2628             float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2629
2630             //  $O_{new} = \exp(m_{old} - m_{new}) * O_{old} + P@V$  (fused multiply-add)
2631             t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
2632             t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
2633             t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
2634             t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
2635
2636             HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2637             HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2638         }
2639
2640         // Update l:  $l_{new} = \exp(m_{old} - m_{new}) * l_{old} + row\_sum(P)$ 

```

```

2641     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
2642     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
2643     lane_block_row_sum_old[i][0] = __fmaf_rn(
2644         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
2645     lane_block_row_sum_old[i][1] = __fmaf_rn(
2646         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
2647
2648     // Update m
2649     lane_block_row_max_old[i][0] = block_row_max_new_0;
2650     lane_block_row_max_old[i][1] = block_row_max_new_1;
2651 }
2652
2653 // Wait for next K tile to be ready in smem before next iteration
2654 if constexpr (kStage > 1) {
2655     if ((tile_K_seqlen + 1) < Tc) {
2656         CP_ASYNC_WAIT_GROUP(0);
2657     }
2658     __syncthreads();
2659 }
2660 } // end outer loop over K seqlen
2661 __syncthreads();
2662
2663 // =====
2664 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
2665 // =====
2666 #pragma unroll
2667 for (int i = 0; i < kValTileSeqLenP; ++i) {
2668     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
2669     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
2670 #pragma unroll
2671     for (int j = 0; j < kValTileHeadDimV; ++j) {
2672         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2673         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2674         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
2675         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
2676         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
2677         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
2678         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2679         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2680     }
2681 }
2682
2683 // =====
2684 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
2685 // =====
2686 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
2687 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
2688 #pragma unroll
2689 for (int i = 0; i < kValTileSeqLenP; ++i) {
2690 #pragma unroll
2691     for (int j = 0; j < kValTileHeadDimV; ++j) {
2692         uint32_t R_Z[2][4];
2693         R_Z[0][0] = R_D[i][j][0];
2694         R_Z[1][0] = R_D[i][j][1];
2695         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
2696         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
2697         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
2698         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
2699         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
2700         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
2701
2702         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
2703         if (lane_id % 4 == 0) {

```

```

2704     int store_warp_regs_0_Br =
2705         warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
2706     int store_lane_gmem_0_Br =
2707         Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
2708     int store_warp_regs_0_d =
2709         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2710     int store_lane_gmem_0_d = store_warp_regs_0_d;
2711     int store_gmem_0_addr_0 = 0_gmem_offset +
2712         (store_lane_gmem_0_Br + 0) * kHeadDim +
2713         store_lane_gmem_0_d;
2714     int store_gmem_0_addr_1 = 0_gmem_offset +
2715         (store_lane_gmem_0_Br + 8) * kHeadDim +
2716         store_lane_gmem_0_d;
2717     // LDST128BITS = reinterpret_cast<float4*>
2718     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
2719         *reinterpret_cast<float4*>(&R_Z[0][0]);
2720     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
2721         *reinterpret_cast<float4*>(&R_Z[1][0]);
2722 }
2723 }
2724 }
2725 }
2726
2727 // =====
2728 // 以下是测试代码，验证 Phase 1 - Phase 8 的kernel的正确性，不评估性能。
2729 // =====
2730
2731 static inline void check(cudaError_t err, const char *msg) {
2732     if (err != cudaSuccess) {
2733         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
2734         exit(EXIT_FAILURE);
2735     }
2736 }
2737
2738
2739 static void test_block_reduce(int N) {
2740
2741     srand(42);
2742     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2743     for (int i = 0; i < N; i++)
2744         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2745
2746     // CPU reference: sum of all elements
2747     double ref = 0.0;
2748     for (int i = 0; i < N; i++) ref += (double)h_a[i];
2749
2750     float *d_a, *d_y;
2751     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
2752     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
2753
2754     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
2755     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
2756
2757     dim3 block(128);
2758     dim3 grid((N + 127) / 128);
2759     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
2760     check(cudaGetLastError(), "blockreduce launch");
2761     check(cudaDeviceSynchronize(), "blockreduce sync");
2762
2763     float result;
2764     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
2765
2766     float err = fabsf(result - (float)ref);

```



```

2767     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
2768           err < 1e-2f ? "PASS" : "FAIL");
2769
2770     free(h_a);
2771     cudaFree(d_a); cudaFree(d_y);
2772 }
2773
2774
2775 static void test_dot(int N) {
2776
2777     srand(42);
2778     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2779     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2780     for (int i = 0; i < N; i++) {
2781         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2782         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2783     }
2784
2785     // CPU reference
2786     double ref = 0.0;
2787     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
2788
2789     float *d_a, *d_b, *d_y;
2790     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
2791     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
2792     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
2793
2794     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
2795     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
2796     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
2797
2798     dim3 block(128);
2799     dim3 grid((N + 127) / 128);
2800     dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
2801     check(cudaGetLastError(), "dot launch");
2802     check(cudaDeviceSynchronize(), "dot sync");
2803
2804     float result;
2805     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
2806
2807     float err = fabsf(result - (float)ref);
2808     printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
2809           err < 1e-2f ? "PASS" : "FAIL");
2810
2811     // ---- Dot Vec4 ----
2812     check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
2813     dim3 block_v4(32);
2814     dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
2815     check(cudaGetLastError(), "dot_vec4 launch");
2816     check(cudaDeviceSynchronize(), "dot_vec4 sync");
2817
2818     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
2819     float err_v4 = fabsf(result - (float)ref);
2820     printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
2821
2822     free(h_a); free(h_b);
2823     cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
2824 }
2825
2826
2827 static void test_relu(int N) {
2828     srand(42);
2829     float *h_x = (float *)malloc((size_t)N * sizeof(float));

```

```

2830 float *h_y = (float *)malloc((size_t)N * sizeof(float));
2831 for (int i = 0; i < N; i++)
2832     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2833
2834 float *d_x, *d_y;
2835 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
2836 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
2837 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
2838
2839 for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
2840 dim3 block256(256);
2841 dim3 grid256((N + 255) / 256);
2842 relu<<<grid256, block256>>>(d_x, d_y, N);
2843 check(cudaGetLastError(), "relu launch");
2844 check(cudaDeviceSynchronize(), "relu sync");
2845 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
2846 float max_err = 0.0f;
2847 for (int i = 0; i < N; i++) {
2848     float expected = fmaxf(0.0f, h_x[i]);
2849     float err = fabsf(h_y[i] - expected);
2850     if (err > max_err) max_err = err;
2851 }
2852 printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2853
2854 dim3 block64(64);
2855 relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
2856 check(cudaGetLastError(), "relu_vec4 launch");
2857 check(cudaDeviceSynchronize(), "relu_vec4 sync");
2858 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
2859 max_err = 0.0f;
2860 for (int i = 0; i < N; i++) {
2861     float expected = fmaxf(0.0f, h_x[i]);
2862     float err = fabsf(h_y[i] - expected);
2863     if (err > max_err) max_err = err;
2864 }
2865 printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2866
2867 free(h_x); free(h_y);
2868 cudaFree(d_x); cudaFree(d_y);
2869 }
2870
2871
2872 static void test_elementwise(int N) {
2873     srand(42);
2874     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2875     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2876     for (int i = 0; i < N; i++) {
2877         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2878         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2879     }
2880
2881     float *d_a, *d_b, *d_c;
2882     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
2883     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
2884     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
2885     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
2886     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
2887
2888     dim3 block256(256);
2889     dim3 grid256((N + 255) / 256);
2890     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
2891     check(cudaGetLastError(), "eadd launch");
2892     check(cudaDeviceSynchronize(), "eadd sync");

```

```

2893 float *h_c = (float *)malloc((size_t)N * sizeof(float));
2894 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
2895 float max_err = 0.0f;
2896 for (int i = 0; i < N; i++) {
2897     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2898     if (err > max_err) max_err = err;
2899 }
2900 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2901
2902 dim3 block64(64);
2903 check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
2904 elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
2905 check(cudaGetLastError(), "eadd_vec4 launch");
2906 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
2907 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
2908 max_err = 0.0f;
2909 for (int i = 0; i < N; i++) {
2910     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2911     if (err > max_err) max_err = err;
2912 }
2913 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2914
2915 free(h_a); free(h_b); free(h_c);
2916 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
2917 }
2918
2919
2920 static void test_histogram(int N) {
2921     srand(42);
2922     int BINS = 16;
2923     int *h_hist = (int *)calloc(BINS, sizeof(int));
2924     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
2925     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
2926     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
2927     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
2928
2929     int *d_idx, *d_hist;
2930     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
2931     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
2932     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
2933     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
2934
2935     dim3 block256(256);
2936     dim3 grid256((N + 255) / 256);
2937     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
2938     check(cudaGetLastError(), "histogram launch");
2939     check(cudaDeviceSynchronize(), "histogram sync");
2940     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
2941
2942     float max_err = 0.0f;
2943     for (int i = 0; i < BINS; i++) {
2944         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
2945         if (err > max_err) max_err = err;
2946     }
2947     printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2948
2949     free(h_hist); free(h_hist_ref); free(h_idx);
2950     cudaFree(d_idx); cudaFree(d_hist);
2951 }
2952
2953
2954 static void test_softmax(int N) {
2955     // Use N = blockDim.x so one block processes all elements as one token.

```

```

2956 constexpr int NUM_THREADS = 256;
2957 if (N != NUM_THREADS) {
2958     printf("    OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
2959     return;
2960 }
2961
2962 srand(42);
2963 float *h_x = (float *)malloc((size_t)N * sizeof(float));
2964 float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
2965 for (int i = 0; i < N; i++)
2966     h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
2967
2968 // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
2969 float max_val = -FLT_MAX;
2970 for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
2971 double sum_exp = 0.0;
2972 for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
2973 for (int i = 0; i < N; i++)
2974     h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
2975
2976 float *d_x, *d_y;
2977 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
2978 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
2979
2980 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
2981
2982 dim3 block(256);
2983 dim3 grid(1); // one token covering all N elements
2984 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2985 check(cudaGetLastError(), "softmax launch");
2986 check(cudaDeviceSynchronize(), "softmax sync");
2987
2988 float *h_y = (float *)malloc((size_t)N * sizeof(float));
2989 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
2990
2991 float max_err = 0.0f;
2992 for (int i = 0; i < N; i++) {
2993     float err = fabsf(h_y[i] - h_y_ref[i]);
2994     if (err > max_err) max_err = err;
2995 }
2996 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2997
2998 // ---- Safe Softmax ----
2999 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3000 check(cudaGetLastError(), "safe_softmax launch");
3001 check(cudaDeviceSynchronize(), "safe_softmax sync");
3002 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
3003 max_err = 0.0f;
3004 for (int i = 0; i < N; i++) {
3005     float err = fabsf(h_y[i] - h_y_ref[i]);
3006     if (err > max_err) max_err = err;
3007 }
3008 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3009
3010 // ---- Naive Softmax ----
3011 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
3012 check(cudaGetLastError(), "naive_softmax launch");
3013 check(cudaDeviceSynchronize(), "naive_softmax sync");
3014 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
3015 max_err = 0.0f;
3016 for (int i = 0; i < N; i++) {
3017     float err = fabsf(h_y[i] - h_y_ref[i]);
3018     if (err > max_err) max_err = err;

```

```

3019 }
3020 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3021
3022 free(h_x); free(h_y); free(h_y_ref);
3023 cudaFree(d_x); cudaFree(d_y);
3024 }
3025
3026
3027 static void test_rms_norm(int N, int K) {
3028
3029     srand(42);
3030     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
3031     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
3032     for (int i = 0; i < N * K; i++)
3033         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3034     float g = 1.5f; // gain
3035
3036     // CPU reference: y = (x / rms(x)) * g
3037     float epsilon = 1e-5f;
3038     for (int n = 0; n < N; n++) {
3039         double sum_sq = 0.0;
3040         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
3041         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
3042         for (int k = 0; k < K; k++)
3043             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
3044     }
3045
3046     float *d_x, *d_y;
3047     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
3048     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
3049     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
3050
3051     dim3 block(128);
3052     dim3 grid(N);
3053     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
3054     check(cudaGetLastError(), "rmsnorm launch");
3055     check(cudaDeviceSynchronize(), "rmsnorm sync");
3056
3057     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
3058     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
3059
3060     float max_err = 0.0f;
3061     for (int i = 0; i < N * K; i++) {
3062         float err = fabsf(h_y[i] - h_y_ref[i]);
3063         if (err > max_err) max_err = err;
3064     }
3065     printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3066
3067     // ---- RMS Norm Vec4 ----
3068     dim3 block_rv4(32);
3069     rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
3070     check(cudaGetLastError(), "rmsnorm_vec4 launch");
3071     check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
3072     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
3073     max_err = 0.0f;
3074     for (int i = 0; i < N * K; i++) {
3075         float err = fabsf(h_y[i] - h_y_ref[i]);
3076         if (err > max_err) max_err = err;
3077     }
3078     printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3079
3080     free(h_x); free(h_y); free(h_y_ref);
3081     cudaFree(d_x); cudaFree(d_y);

```

```

3082 }
3083
3084
3085 static void test_layer_norm(int N, int K) {
3086
3087     srand(42);
3088     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
3089     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
3090     for (int i = 0; i < N * K; i++)
3091         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3092     float g = 1.5f, b = 0.3f; // gain and bias
3093
3094     // CPU reference: y = ((x - mean) / std) * g + b
3095     float epsilon = 1e-5f;
3096     for (int n = 0; n < N; n++) {
3097         double sum = 0.0;
3098         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
3099         float mean = (float)sum / (float)K;
3100         double sum_sq = 0.0;
3101         for (int k = 0; k < K; k++) {
3102             float diff = h_x[n * K + k] - mean;
3103             sum_sq += (double)diff * (double)diff;
3104         }
3105         float std = sqrtf((float)sum_sq / (float)K + epsilon);
3106         for (int k = 0; k < K; k++)
3107             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
3108     }
3109
3110     float *d_x, *d_y;
3111     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
3112     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
3113     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
3114
3115     dim3 block(128);
3116     dim3 grid(N);
3117     layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
3118     check(cudaGetLastError(), "layernorm launch");
3119     check(cudaDeviceSynchronize(), "layernorm sync");
3120
3121     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
3122     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
3123
3124     float max_err = 0.0f;
3125     for (int i = 0; i < N * K; i++) {
3126         float err = fabsf(h_y[i] - h_y_ref[i]);
3127         if (err > max_err) max_err = err;
3128     }
3129     printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3130
3131     // ---- Layer Norm Vec4 ----
3132     dim3 block_lv4(32);
3133     layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
3134     check(cudaGetLastError(), "layernorm_vec4 launch");
3135     check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
3136     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
3137     max_err = 0.0f;
3138     for (int i = 0; i < N * K; i++) {
3139         float err = fabsf(h_y[i] - h_y_ref[i]);
3140         if (err > max_err) max_err = err;
3141     }
3142     printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3143
3144     free(h_x); free(h_y); free(h_y_ref);

```

```

3145     cudaFree(d_x); cudaFree(d_y);
3146 }
3147
3148
3149 static void test_rope(int seq_len, int N) {
3150
3151     int total_pairs = seq_len * N;
3152     int total_elems = total_pairs * 2;
3153     size_t size = (size_t)total_elems * sizeof(float);
3154
3155     float *h_x = (float *)malloc(size);
3156     float *h_y_ref = (float *)malloc(size);
3157
3158     srand(42);
3159     for (int i = 0; i < total_elems; i++)
3160         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3161
3162     // CPU reference: 2D rotation for each pair
3163     for (int idx = 0; idx < total_pairs; idx++) {
3164         int token_pos = idx / N;
3165         int token_idx = idx % N;
3166         float x1 = h_x[idx * 2];
3167         float x2 = h_x[idx * 2 + 1];
3168         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
3169         float angle = (float)token_pos * theta;
3170         float cos_v = cosf(angle);
3171         float sin_v = sinf(angle);
3172         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
3173         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
3174     }
3175
3176     float *d_x, *d_y;
3177     check(cudaMalloc(&d_x, size), "rope alloc X");
3178     check(cudaMalloc(&d_y, size), "rope alloc Y");
3179     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
3180
3181     dim3 block(256);
3182     dim3 grid((total_pairs + 255) / 256);
3183     rope<<<grid, block>>>(d_x, d_y, seq_len, N);
3184     check(cudaGetLastError(), "rope launch");
3185     check(cudaDeviceSynchronize(), "rope sync");
3186
3187     float *h_y = (float *)malloc(size);
3188     check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
3189
3190     float max_err = 0.0f;
3191     for (int i = 0; i < total_elems; i++) {
3192         float err = fabsf(h_y[i] - h_y_ref[i]);
3193         if (err > max_err) max_err = err;
3194     }
3195     printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3196
3197     free(h_x);
3198     free(h_y);
3199     free(h_y_ref);
3200     cudaFree(d_x);
3201     cudaFree(d_y);
3202 }
3203
3204
3205 static void test_mat_transpose(int row, int col) {
3206
3207     size_t size_in = (size_t)row * col * sizeof(float);

```



```

3208 size_t size_out = (size_t)col * row * sizeof(float);
3209
3210 float *h_x = (float *)malloc(size_in);
3211 float *h_y_ref = (float *)malloc(size_out);
3212
3213 srand(42);
3214 for (int i = 0; i < row * col; i++)
3215     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3216
3217 // CPU reference: y[j][i] = x[i][j] (row-major)
3218 for (int i = 0; i < row; i++)
3219     for (int j = 0; j < col; j++)
3220         h_y_ref[j * row + i] = h_x[i * col + j];
3221
3222 float *d_x, *d_y;
3223 check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
3224 check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
3225 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
3226
3227 dim3 block(16, 16);
3228 dim3 grid((col + 15) / 16, (row + 15) / 16);
3229 mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
3230 check(cudaGetLastError(), "mattrans launch");
3231 check(cudaDeviceSynchronize(), "mattrans sync");
3232
3233 float *h_y = (float *)malloc(size_out);
3234 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
3235
3236 float max_err = 0.0f;
3237 for (int i = 0; i < col * row; i++) {
3238     float err = fabsf(h_y[i] - h_y_ref[i]);
3239     if (err > max_err) max_err = err;
3240 }
3241 printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3242
3243 free(h_x);
3244 free(h_y);
3245 free(h_y_ref);
3246 cudaFree(d_x);
3247 cudaFree(d_y);
3248 }
3249
3250
3251 static void test_mat_transpose_padded(int row, int col) {
3252
3253     size_t size_in = (size_t)row * col * sizeof(float);
3254     size_t size_out = (size_t)col * row * sizeof(float);
3255
3256     float *h_x = (float *)malloc(size_in);
3257     float *h_y_ref = (float *)malloc(size_out);
3258
3259     srand(42);
3260     for (int i = 0; i < row * col; i++)
3261         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3262
3263     // CPU reference: y[j][i] = x[i][j] (row-major)
3264     for (int i = 0; i < row; i++)
3265         for (int j = 0; j < col; j++)
3266             h_y_ref[j * row + i] = h_x[i * col + j];
3267
3268     float *d_x, *d_y;
3269     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
3270     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");

```

```

3271 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
3272
3273 dim3 block(16, 16);
3274 dim3 grid((col + 15) / 16, (row + 63) / 64);
3275 mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
3276 check(cudaGetLastError(), "mattrans_padded launch");
3277 check(cudaDeviceSynchronize(), "mattrans_padded sync");
3278
3279 float *h_y = (float *)malloc(size_out);
3280 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
3281
3282 float max_err = 0.0f;
3283 for (int i = 0; i < col * row; i++) {
3284     float err = fabsf(h_y[i] - h_y_ref[i]);
3285     if (err > max_err) max_err = err;
3286 }
3287 printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3288
3289 free(h_x);
3290 free(h_y);
3291 free(h_y_ref);
3292 cudaFree(d_x);
3293 cudaFree(d_y);
3294 }
3295
3296
3297 static void test_sgemv(int M, int K) {
3298
3299     srand(42);
3300     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
3301     float *h_x = (float *)malloc((size_t)K * sizeof(float));
3302     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3303     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3304     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3305
3306     // CPU reference: y[m] = sum_k A[m,k] * x[k]
3307     for (int m = 0; m < M; m++) {
3308         double sum = 0.0;
3309         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
3310         h_y_ref[m] = (float)sum;
3311     }
3312
3313     float *d_a, *d_x, *d_y;
3314     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
3315     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
3316     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
3317
3318     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
3319     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
3320
3321     dim3 block(32, 4);
3322     dim3 grid((M + 3) / 4);
3323     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
3324     check(cudaGetLastError(), "sgemv launch");
3325     check(cudaDeviceSynchronize(), "sgemv sync");
3326
3327     float *h_y = (float *)malloc((size_t)M * sizeof(float));
3328     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
3329
3330     float max_err = 0.0f;
3331     for (int m = 0; m < M; m++) {
3332         float err = fabsf(h_y[m] - h_y_ref[m]);
3333         if (err > max_err) max_err = err;

```

```

3334 }
3335 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3336
3337 // ---- SGEMV K32 ----
3338 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
3339 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
3340 check(cudaGetLastError(), "sgemv_k32 launch");
3341 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
3342
3343 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
3344
3345 max_err = 0.0f;
3346 for (int m = 0; m < M; m++) {
3347     float err = fabsf(h_y[m] - h_y_ref[m]);
3348     if (err > max_err) max_err = err;
3349 }
3350 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3351
3352 // ---- SGEMV K16 ----
3353 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3354 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3355
3356 int K16 = 16;
3357 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
3358 h_x = (float *)malloc((size_t)K16 * sizeof(float));
3359 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3360 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3361 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3362
3363 for (int m = 0; m < M; m++) {
3364     double sum = 0.0;
3365     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
3366     h_y_ref[m] = (float)sum;
3367 }
3368
3369 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
3370 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
3371 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
3372
3373 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
3374 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
3375
3376 dim3 grid_k16((M + 7) / 8);
3377 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
3378 check(cudaGetLastError(), "sgemv_k16 launch");
3379 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
3380
3381 h_y = (float *)malloc((size_t)M * sizeof(float));
3382 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
3383
3384 max_err = 0.0f;
3385 for (int m = 0; m < M; m++) {
3386     float err = fabsf(h_y[m] - h_y_ref[m]);
3387     if (err > max_err) max_err = err;
3388 }
3389 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3390
3391 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3392 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3393 }
3394
3395
3396 static void test_sgemm(int M, int N, int K) {

```

```

3397
3398 size_t size_a = (size_t)M * K * sizeof(float);
3399 size_t size_b = (size_t)K * N * sizeof(float);
3400 size_t size_c = (size_t)M * N * sizeof(float);
3401
3402 float *h_a = (float *)malloc(size_a);
3403 float *h_b = (float *)malloc(size_b);
3404 float *h_c_ref = (float *)malloc(size_c);
3405
3406 srand(42);
3407 for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3408 for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3409
3410 float *d_a, *d_b, *d_c;
3411 check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
3412 check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
3413 check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
3414
3415 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
3416 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
3417
3418 // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
3419 cublasHandle_t handle;
3420 cublasCreate(&handle);
3421 float alpha = 1.0f, beta = 0.0f;
3422 cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3423             &alpha, d_b, N, d_a, K, &beta, d_c, N);
3424 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
3425
3426 // Kernel
3427 cudaEvent_t start, stop;
3428 cudaEventCreate(&start);
3429 cudaEventCreate(&stop);
3430
3431 dim3 block(32, 32);
3432 dim3 grid((N + 31) / 32, (M + 31) / 32);
3433 sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
3434 check(cudaGetLastError(), "sgemm launch");
3435 check(cudaDeviceSynchronize(), "sgemm sync");
3436
3437 float *h_c = (float *)malloc(size_c);
3438 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
3439
3440 // Verify
3441 float max_err = 0.0f;
3442 for (int i = 0; i < M * N; i++) {
3443     float err = fabsf(h_c[i] - h_c_ref[i]);
3444     if (err > max_err) max_err = err;
3445 }
3446 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3447
3448 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
3449
3450 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
3451 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
3452 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
3453 check(cudaGetLastError(), "sgemm_vec4 launch");
3454 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
3455
3456 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
3457
3458 max_err = 0.0f;
3459 for (int i = 0; i < M * N; i++) {

```

```

3460     float err = fabsf(h_c[i] - h_c_ref[i]);
3461     if (err > max_err) max_err = err;
3462 }
3463 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3464
3465 free(h_a); free(h_b); free(h_c); free(h_c_ref);
3466 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3467 cublasDestroy(handle);
3468 cudaEventDestroy(start);
3469 cudaEventDestroy(stop);
3470 }
3471
3472
3473 static void test_hgemm_mma(int M, int N, int K) {
3474
3475     size_t size_a = (size_t)M * K * sizeof(half);
3476     size_t size_b = (size_t)K * N * sizeof(half);
3477     size_t size_c = (size_t)M * N * sizeof(half);
3478
3479     half *h_a = (half *)malloc(size_a);
3480     half *h_b = (half *)malloc(size_b);
3481     half *h_c_ref = (half *)malloc(size_c);
3482
3483     srand(42);
3484     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3485     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3486
3487     // Kernel expects B^T [N×K] row-major layout (TN layout convention).
3488     // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
3489     size_t size_b_t = (size_t)N * K * sizeof(half);
3490     half *h_b_t = (half *)malloc(size_b_t);
3491     for (int n = 0; n < N; n++)
3492         for (int k = 0; k < K; k++)
3493             h_b_t[n * K + k] = h_b[k * N + n];
3494
3495     half *d_a, *d_b, *d_b_t, *d_c;
3496     check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
3497     check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
3498     check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
3499     check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
3500
3501     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
3502     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
3503     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
3504
3505     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
3506     // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
3507     cublasHandle_t handle;
3508     cublasCreate(&handle);
3509     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3510     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3511                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
3512                 &beta_h, d_c, CUDA_R_16F, N,
3513                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3514     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
3515
3516     // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
3517     cudaEvent_t start, stop;
3518     cudaEventCreate(&start);
3519     cudaEventCreate(&stop);
3520
3521     constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
3522     size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576

```

```

3523 dim3 block(256);
3524 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3525 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
3526 check(cudaGetLastError(), "hgemm launch");
3527 check(cudaDeviceSynchronize(), "hgemm sync");
3528
3529 half *h_c = (half *)malloc(size_c);
3530 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
3531
3532 // Verify
3533 float max_err = 0.0f;
3534 for (int i = 0; i < M * N; i++) {
3535     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3536     if (err > max_err) max_err = err;
3537 }
3538 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
3539
3540 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
3541 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
3542 cublasDestroy(handle);
3543 cudaEventDestroy(start);
3544 cudaEventDestroy(stop);
3545 }
3546
3547
3548 #if defined(NOTES_V2_HAS_WGMMA) || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) || defined(
    __CUDA_ARCH_FEAT_SM90_ALL)
3549 static void test_hgemm_wgmma(int M, int N, int K) {
3550     // HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
3551     // TN layout: C[M×N] = A[M×K] × BT[N×K]
3552     // Kernel: hgemm_wgmma_stages_tn with default template params
3553
3554     constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
3555
3556     // M, K must be divisible by tile dims
3557     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
3558         printf("| %-35s | %-12s | %-4s |\n",
3559             "HGEMM WGMMA", "SKIP", "SKIP");
3560         return;
3561     }
3562
3563     size_t size_a = (size_t)M * K * sizeof(half);
3564     size_t size_b = (size_t)K * N * sizeof(half);
3565     size_t size_c = (size_t)M * N * sizeof(half);
3566
3567     half *h_a = (half *)malloc(size_a);
3568     half *h_b = (half *)malloc(size_b);
3569     half *h_c_ref = (half *)malloc(size_c);
3570
3571     srand(42);
3572     for (int i = 0; i < M * K; i++)
3573         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3574     for (int i = 0; i < K * N; i++)
3575         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3576
3577     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
3578     size_t size_b_t = (size_t)N * K * sizeof(half);
3579     half *h_b_t = (half *)malloc(size_b_t);
3580     for (int n = 0; n < N; n++)
3581         for (int k = 0; k < K; k++)
3582             h_b_t[n * K + k] = h_b[k * N + n];
3583
3584     half *d_a, *d_b, *d_b_t, *d_c;

```

```

3585 check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
3586 check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
3587 check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
3588 check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
3589
3590 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
3591 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
3592        "wgmma H2D B (cuBLAS)");
3593 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
3594        "wgmma H2D B_t (kernel)");
3595
3596 // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
3597 cublasHandle_t handle;
3598 cublasCreate(&handle);
3599 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3600 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
3601              CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
3602              CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3603 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
3604        "wgmma D2H ref");
3605
3606 // Create TMA tensor maps for A and B^T
3607 // A[MxK] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
3608 // B^T[NxK] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
3609 CUTensorMap *tma_a =
3610     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
3611 CUTensorMap *tma_b =
3612     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
3613
3614 // Launch WGMMMA kernel
3615 // BLOCK_SWIZZLE=false → 2D grid, no swizzle
3616 size_t smem_bytes =
3617     K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
3618 cudaFuncSetAttribute(
3619     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
3620     false>,
3621     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
3622
3623 dim3 block(NUM_THREADS);
3624 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3625 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
3626     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
3627 check(cudaGetLastError(), "wgmma launch");
3628 check(cudaDeviceSynchronize(), "wgmma sync");
3629
3630 half *h_c = (half *)malloc(size_c);
3631 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
3632
3633 // Verify
3634 float max_err = 0.0f;
3635 for (int i = 0; i < M * N; i++) {
3636     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3637     if (err > max_err)
3638         max_err = err;
3639 }
3640 printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
3641        max_err < 1.0f ? "PASS" : "FAIL");
3642
3643 free(h_a);
3644 free(h_b);
3645 free(h_b_t);
3646 free(h_c);
3647 free(h_c_ref);

```



```

3648 cudaFree(d_a);
3649 cudaFree(d_b);
3650 cudaFree(d_b_t);
3651 cudaFree(d_c);
3652 cudaFree(tma_a);
3653 cudaFree(tma_b);
3654 cublasDestroy(handle);
3655 }
3656 #endif /* NOTES_V2_HAS_WGMMA */
3657
3658
3659 static void test_flash_attn(int seqlen, int head_dim) {
3660     // FlashAttention-2 with split-Q, MMA m16n8k16
3661     int B = 1, H = 8;
3662
3663     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
3664
3665     srand(42);
3666     half *h_q = (half *)malloc(sz);
3667     half *h_k = (half *)malloc(sz);
3668     half *h_v = (half *)malloc(sz);
3669     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
3670         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3671         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3672         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3673     }
3674
3675     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
3676     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
3677     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
3678     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
3679     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
3680     int count = B * H * seqlen * head_dim;
3681     for (int i = 0; i < count; i++) {
3682         ref_q[i] = __half2float(h_q[i]);
3683         ref_k[i] = __half2float(h_k[i]);
3684         ref_v[i] = __half2float(h_v[i]);
3685     }
3686
3687     float scale = 1.0f / sqrtf((float)head_dim);
3688     for (int bi = 0; bi < B * H; bi++) {
3689         for (int qi = 0; qi < seqlen; qi++) {
3690             // S[qi, kj] = Q[qi,:] @ K[kj,:]^T * scale
3691             float smax = -INFINITY;
3692             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
3693             for (int kj = 0; kj < seqlen; kj++) {
3694                 float s = 0.0f;
3695                 for (int d = 0; d < head_dim; d++)
3696                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
3697                        ref_k[bi * seqlen * head_dim + kj * head_dim + d];
3698                 S[kj] = s * scale;
3699                 if (S[kj] > smax) smax = S[kj];
3700             }
3701             // softmax
3702             double sum_exp = 0.0;
3703             for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
3704             float inv_sum = 1.0f / (float)sum_exp;
3705             // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
3706             for (int d = 0; d < head_dim; d++) {
3707                 double o_acc = 0.0;
3708                 for (int kj = 0; kj < seqlen; kj++)
3709                     o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
3710                        ref_v[bi * seqlen * head_dim + kj * head_dim + d];

```

```

3711         ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
3712     }
3713     free(S);
3714 }
3715 }
3716
3717 half *d_q, *d_k, *d_v, *d_o;
3718 check(cudaMalloc(&d_q, sz), "fa alloc Q");
3719 check(cudaMalloc(&d_k, sz), "fa alloc K");
3720 check(cudaMalloc(&d_v, sz), "fa alloc V");
3721 check(cudaMalloc(&d_o, sz), "fa alloc O");
3722 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
3723 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
3724 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
3725
3726 // Template params for kHeadDim=64, kStage=2
3727 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
3728 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
3729 constexpr int kMmaTileSeqLenQ = 4;
3730 constexpr int kMmaTileSeqLenK = 1;
3731 constexpr int kMmaTileSeqLenP = 4;
3732 constexpr int kMmaTileHeadDimV = 1;
3733 constexpr int kValTileSeqLenQ = 1;
3734 constexpr int kValTileSeqLenK = 8;
3735 constexpr int kValTileSeqLenP = 1;
3736 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
3737
3738 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
3739 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
3740 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
3741                     kStageV * Bc * (kHeadDim + kPadV) +
3742                     Bc * (kHeadDim + kPadV)) * sizeof(half);
3743
3744 dim3 block(128);
3745 dim3 grid((seqlen + Br - 1) / Br, B * H);
3746
3747 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
3748 kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
3749 kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
3750 kStageV, kPadV>
3751 <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
3752 check(cudaGetLastError(), "fa launch");
3753 check(cudaDeviceSynchronize(), "fa sync");
3754
3755 half *h_o = (half *)malloc(sz);
3756 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
3757
3758 float max_err = 0.0f;
3759 for (int i = 0; i < count; i++) {
3760     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
3761     if (err > max_err) max_err = err;
3762 }
3763 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
3764
3765 free(h_q); free(h_k); free(h_v); free(h_o);
3766 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
3767 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
3768 }
3769
3770
3771 int main(int argc, char *argv[]) {
3772 #if defined(NOTES_V2_HAS_WGMMMA) || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) || defined(
    __CUDA_ARCH_FEAT_SM90_ALL)

```

```

3773     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
3774 #endif
3775     int M = 1024, N = 1024, K = 1024;
3776     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
3777
3778     printf("=== notes-v2.cu verification harness ===\n");
3779     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
3780     printf("|-----|-----|-----|\n");
3781
3782     test_block_reduce(N);
3783     test_dot(N);
3784     test_relu(1024);
3785     test_elementwise(1024);
3786     test_histogram(1024);
3787     test_softmax(256); // softmax kernel requires N == blockDim.x
3788     test_rms_norm(8, 128);
3789     test_layer_norm(8, 128);
3790     test_rope(8, 128);
3791     test_mat_transpose(256, 256);
3792     test_mat_transpose_padded(256, 256);
3793     test_sgemv(256, 128);
3794     test_sgemm(M, N, K);
3795     test_hgemm_mma(M, N, K);
3796     #if (defined(NOTES_V2_HAS_WGMMA) \
3797         || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) \
3798         || defined(__CUDA_ARCH_FEAT_SM90_ALL))
3799         test_hgemm_wgmma(M, N, K);
3800     #endif
3801     test_flash_attn(1024, 64);
3802
3803     printf("=== All tests done ===\n");
3804     return 0;
3805 }
3806
3807 // =====
3808 // Quick build & run reference
3809 // =====
3810 // # sm_89 单独编译 + 运行:
3811 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
3812 //
3813 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
3814 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a -DNOTES_V2_HAS_WGMMA -lcublas -lcuda notes-v2.
    cu -o notes_v2_sm90.bin

```